

Notas sobre Métodos Numéricos con R

Carlos E. Martínez-Rodríguez
Universidad Autónoma de la Ciudad de México
Academia de Matemáticas
`carlos.martinez@uacm.edu.mx`

Índice

1. Sobre el curso	3
1.1. Requisitos para acreditar la materia	3
1.2. De la naturaleza del curso	4
2. Introducción	4
3. Revisión de temas importantes	4
3.1. Cálculo	4
3.1.1. Límites y continuidad	5
3.1.2. Continuidad y convergencia de sucesiones	5
3.1.3. Derivabilidad	6
3.1.4. Integración	7
3.1.5. Polinomios de Taylor	8
3.1.6. Teoremas adicionales	9
3.2. Álgebra Lineal	10
3.2.1. Matrices	10
3.2.2. Operaciones con matrices	10
3.2.3. Matrices especiales	11
3.2.4. Determinante de una matriz	11
3.2.5. Valores propios y vectores propios	12
3.2.6. Normas vectoriales y normas matriciales	12
4. Operaciones de punto flotante	13
4.1. Sistemas decimal y binario	13
4.2. Números en punto flotante	15
4.3. Representación	16
4.4. Errores	17
4.5. Cuantificación de errores	18
4.6. Errores en punto flotante	19
4.7. Aproximación numérica y errores	20
4.8. Ejercicios y Tareas	21

5. Solución de Sistemas de Ecuaciones Lineales	21
5.1. Eliminación gaussiana simple	21
5.2. Construcción de L y U	22
5.3. Pivoteo y escalamiento	22
5.4. Gauss–Jordan e inversa	22
5.5. Eliminación Gaussiana Simple	22
5.6. Eliminación Gaussiana con Pivoteo Parcial	23
5.7. Eliminación con Pivoteo y Escalamiento	23
5.8. Gauss–Jordan y cálculo de inversas	23
5.9. Sustitución hacia adelante y hacia atrás	24
5.10. Descomposición de A	25
5.11. Esquema general	25
5.12. Método de Jacobi	25
5.13. Método de Gauss–Seidel	26
5.14. Condiciones de convergencia	26
5.15. Método SOR (Successive Over-Relaxation)	26
5.16. Eliminación Gaussiana Simple	26
5.17. Eliminación con pivoteo parcial	27
5.18. Eliminación con pivoteo y escalamiento	27
5.19. Gauss–Jordan e inversas	27
5.20. Descomposición de A	27
5.21. Forma general de iteración	28
5.22. Método de Jacobi	28
5.23. Método de Gauss–Seidel	28
5.24. Método SOR	28
5.25. Convergencia	28
5.26. Resumen con matrices de iteración	29
5.27. Factorización LU	29
5.27.1. Método general (sin permutaciones)	29
5.27.2. Método con permutaciones	30
5.27.3. Resumen	30
5.27.4. Ejemplo 1: $A = LU$ (sin permutaciones)	30
5.27.5. Ejemplo 2: $PA = LU$ (con permutaciones)	31
5.27.6. Ejemplo: Resolver $Ax = b$ vía LU	32
5.28. Definiciones fundamentales de matrices	33
5.29. Factorización de Doolittle	33
5.30. Factorización de Cholesky	33
5.31. Doolittle (LU)	35
5.32. Cholesky (LL^T)	35
5.33. Métodos iterativos para resolver sistemas lineales	37
5.33.1. Descomposición de la matriz	37
5.34. Método de Jacobi	38
5.35. Método de Gauss–Seidel	38
5.36. Condiciones de convergencia	38
5.37. Esquema general	38
5.37.1. Ejemplos	39
5.37.2. Sistema de prueba (diagonalmente dominante)	39
5.37.3. Método de Jacobi	39

5.37.4. Método de Gauss–Seidel	39
5.38. Método de Relajación Sucesiva (SOR)	40
5.39. SOR en una sola iteración (comparación con Gauss–Seidel)	41
5.40. Ejercicios	41
5.40.1. Ejercicios	41
5.40.2. Ejercicios	43
5.40.3. Ejercicios	44
5.40.4. Ejercicios	46
5.40.5. Ejercicios	47
6. Introducción al uso de R	48
6.1. Sesiones en RStudio	48
6.2. Uso de R	49
6.3. Funciones	49
6.4. Clase en Laboratorio de Cómputo	50
7. Apéndice A: Breve historia de los Métodos Numéricos	54
8. Programas y rutinas en R	55
8.1. Operadores lógicos	56
8.2. OPERADORES ARITMETICOS	56
8.2.1. SUMA, RESTA, MULTIPLICACION, DIVISION, POTENCIA, MODULO, DIVISION ENTERA	56
8.2.2. LOGARITMOS Y EXPONENCIALES	56
8.2.3. FUNCIONES TRIGONOMETRICAS	56
8.2.4. FUNCIONES VARIAS	56
8.2.5. EJERCICIOS DE PRACTICA	57
8.3. EJERCICIOS DE PRACTICA	57
8.3.1. DEFINICION DE CONSTANTES	57
8.3.2. CONCATENAR Y PEGAR EXPRESIONES	57
8.3.3. ASIGNACION E IMPRESION	57
8.3.4. LISTADO DE OBJETOS DEFINIDOS	57
8.3.5. IMPRIMIR PEGAR AVANZADO	58
8.3.6. EJERCICIOS DE PRACTICA	58
8.3.7. DEFINICION DE FUNCIONES	58
8.3.8. Ejemplo 1	58
8.3.9. Ejemplo 2	58
8.3.10. GRAFICAS	58
8.3.11. EJEMPLOS DE PRACTICA	58
8.4. Introducción a Factorización LU	59
8.4.1. Inversa de una matriz	59
8.4.2. Factorización LU	61
8.4.3. Notas importantes	65
8.5. Métodos de Eliminación directa	66
8.5.1. Gaussiana Simple	66
8.5.2. Gaussiana con pivoteo parcial	67
8.5.3. Gauss Jordan	68
8.5.4. Cálculo de Inversa	69
8.5.5. Factorización LU	71

8.5.6.	Resolucion con pivoteo	73
8.5.7.	Factorización de Cholesky	74
8.5.8.	Solución vía Cholesky	74
8.6.	Métodos Iterativos	75
8.6.1.	Gauss Jacobi	75
8.6.2.	Gauss Seidel	76

1. Sobre el curso

El curso se llevará a cabo tres veces a la semana, con sesiones de 1.5 horas, de las cuales una de ellas se realizará en el laboratorio de cómputo 3.

1.1. Requisitos para acreditar la materia

El curso por su naturaleza implica que la/el estudiante implemente los distintos algoritmos que se revisan en el curso, por lo tanto es importante que demuestre que efectivamente puede implementar, revisar y mejorar los algoritmos ya existentes.

Hay dos maneras de certificar la materia: a) Portafolio y b) Examen de certificación. Al menos una semana antes de que termine el curso las y los estudiantes tendrán conocimiento de sus calificaciones parciales y por tanto de la calificación promedio obtenida al momento, para que sea el/la mismo(a) estudiante quién decida si certifica por la modalidad de portafolio, o por la modalidad de examen de certificación.

El portafolio se conforma de evaluaciones (40 %), programas, (40 %) tareas (10 %), tareitas (5 %)y trabajos adicionales (5 %). Mientras que la certificación es un examen elaborado por el comité de certificación y que será presentado por todas y todos los estudiantes que se inscriban en esta modalidad en las fechas establecidas por la Coordinación de Certificación y Registro. En cualquiera de las dos modalidades es indispensable que el/la estudiante se registre a este proceso para que su calificación pueda ser asignada al final del proceso de Certificación.

1.2. De la naturaleza del curso

El curso tiene un fuerte sustento en la programación constante, sin embargo, es importante resaltar que los conceptos teóricos deben ser dominados por las y los estudiantes, por lo tanto las evaluaciones y las tareas tendrán estas dos componentes principales. Al contrario de lo que pueda pensarse la asistencia al curso es obligatoria pero no influye directamente en la calificación obtenida. Sin embargo, hay que mencionar que si la asistencia se realiza de manera intermitente es probable que cueste un poco de trabajo reincorporarse a la dinámica de trabajo que se irá construyendo con el grupo con el transcurso de las clases.

2. Introducción

En este curso estudiaremos los elementos básicos de los métodos numéricos, utilizando el programa de distribución libre *R*. Antes de iniciar propiamente con el estudio de los métodos numéricos realizaremos un breve repaso de algunos conceptos de álgebra lineal, cálculo diferencial mismos que son fundamentales en esta materia y de la que se supone las y los estudiantes se encuentran familiarizados con ellos.

Análisis Numérico es una rama de las matemáticas que, mediante el uso de algoritmos iterativos, obtiene soluciones numéricas a problemas en los cuales la matemática simbólica (o analítica) resulta poco eficiente o no puede ofrecer un resultado. En particular, a estos algoritmos se les denomina *métodos numéricos*. Por lo general los métodos numéricos se componen de un número de pasos finitos que se ejecutan de manera lógica, mejorando aproximaciones iniciales a cierta cantidad, tal como la raíz de una ecuación, hasta que se cumple con cierta cota de error. A esta operación cíclica de mejora del valor se le conoce como *iteración*. El análisis numérico es una alternativa muy eficiente para la resolución de ecuaciones, tanto algebraicas (polinomios) como trascendentes teniendo una ventaja muy importante respecto a otro tipo de métodos: La repetición de instrucciones lógicas (iteraciones), proceso que permite mejorar los valores inicialmente considerados como solución. Dado que se trata siempre de la misma operación lógica, resulta muy pertinente el uso de recursos de cómputo para realizar esta tarea. Sin embargo, debe haber claridad en el sentido de que el análisis numérico no es la panacea en la solución de problemas matemáticos; los métodos numéricos arrojan *aproximaciones*, es decir, están sujetos a un error. Esto quiere decir que si se puede ser tan preciso como los recursos de cálculo lo permitan, siempre está presente y debe considerarse su manejo en el desarrollo de las soluciones requeridas. El uso de diversos sistemas de cómputo determina qué soluciones analítico-numéricas son viables en la práctica, lo que implica que se deben tomar en cuenta el proceso iterativo, el costo de los recursos físicos que se emplean en el análisis, y el tipo de práctica de la Ingeniería.

3. Revisión de temas importantes

3.1. Cálculo

Comenzamos el capítulo con un repaso de algunos aspectos importantes del cálculo que son necesarios a lo largo del texto. Suponemos que los estudiantes que lean este texto conocen la terminología, la notación y los resultados que se dan en un curso típico de cálculo.

3.1.1. Límites y continuidad

El límite de una función nos dice qué tan cerca se encuentran las imágenes de una función si dos elementos de su dominio se encuentran lo suficientemente cerca, es decir, decimos que una función $f(x)$ definida en el intervalo (a, b) tiene **límite** L en el punto $x = x_0$, lo que denotamos por

$$\lim_{x \rightarrow x_0} f(x) = L,$$

si para cualquier $\varepsilon > 0$, existe un número real $\delta > 0$ tal que $|f(x) - L| < \varepsilon$ siempre que $0 < |x - x_0| < \delta$. Es decir, que los valores de la función estarán cerca de L siempre que x esté suficientemente cerca de x_0 .

Se dice que una función f es continua en a si cuando x se aproxima al valor de a , entonces también $f(x)$ se aproxima a $f(a)$, es decir, $f(x)$ es **continua** en el punto $x = a$ si

$$\lim_{x \rightarrow a} f(x) = f(a),$$

y se dice que f es **continua en el conjunto** (a, b) si es continua en cada uno de los puntos del intervalo. Denotaremos el conjunto de todas las funciones f que son continuas en $E = (a, b)$ por $C(E)$.

Se dice que una sucesión $\{x_n\}_{n=1}^{\infty}$ **converge** a un número x , si $\lim_{n \rightarrow \infty} x_n = x$ o bien, $x_n \rightarrow x$ cuando $n \rightarrow \infty$, si para cualquier $\varepsilon > 0$, existe un número natural $N(\varepsilon)$ tal que $|x_n - x| < \varepsilon$ para cada $n > N(\varepsilon)$. Cuando una sucesión tiene límite, se dice que la **sucesión converge**.

3.1.2. Continuidad y convergencia de sucesiones

Si $f(x)$ es una función definida en un conjunto S de números reales y $x_0 \in S$, entonces las siguientes afirmaciones son equivalentes:

1. $f(x)$ es continua en $x = x_0$,
2. Si $\{x_n\}_{n=1}^{\infty}$ es cualquier sucesión en S que converge a x_0 , entonces $\lim_{n \rightarrow \infty} f(x_n) = f(x_0)$.

Teorema 1 (Teorema del valor intermedio o de Bolzano.). Si $f \in C[a, b]$ y ℓ es un número cualquiera entre $f(a)$ y $f(b)$, entonces existe al menos un número $c \in (a, b)$ tal que $f(c) = \ell$. Véase la Figura 1.

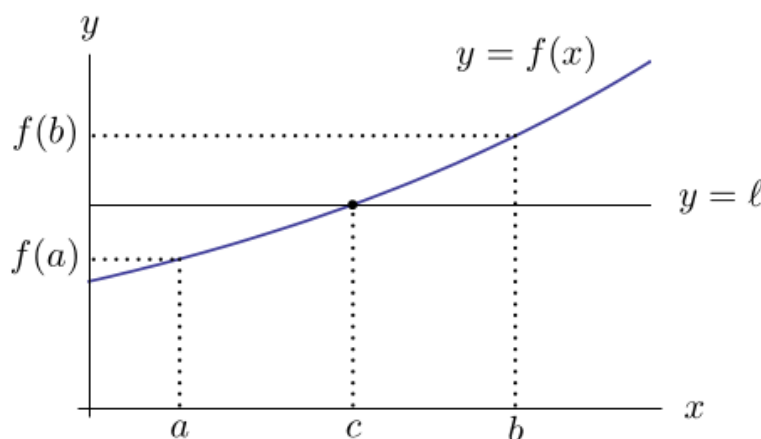


Figura 1: Teorema del valor intermedio o de Bolzano

Nota 1. Todas las funciones con las que se van a trabajar en este curso de métodos numéricos serán continuas, ya que esto es lo mínimo que debemos exigir para asegurar que la conducta de un método se puede predecir.

3.1.3. Derivabilidad

Si $f(x)$ es una función definida en un intervalo abierto que contiene un punto x_0 , entonces se dice que $f(x)$ es **derivable** en $x = x_0$ cuando existe el límite

$$f'(x_0) = \lim_{x \rightarrow x_0} \frac{f(x) - f(x_0)}{x - x_0}.$$

El número $f'(x_0)$ se llama **derivada** de f en x_0 y coincide con la pendiente de la recta tangente a la gráfica de f en el punto $(x_0, f(x_0))$, Figura 2.

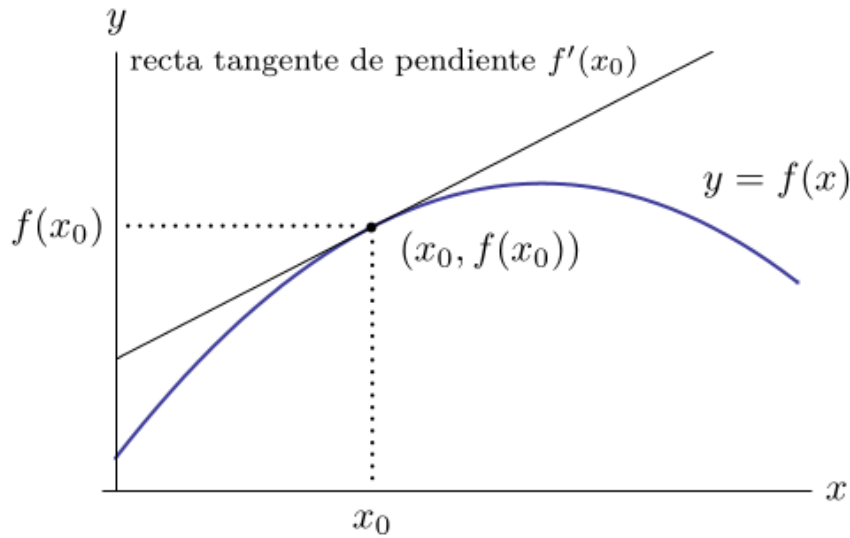


Figura 2: Derivada de una función en un punto

Derivabilidad implica continuidad. Si la función $f(x)$ es derivable en $x = x_0$, entonces $f(x)$ es continua en $x = x_0$. El conjunto de todas las funciones que admiten n derivadas continuas en S se denota por $C^n(S)$, mientras que el conjunto de todas las funciones indefinidamente derivables en S se denota por $C^\infty(S)$. Las funciones polinómicas, racionales, trigonométricas, exponenciales y logarítmicas están en $C^\infty(S)$, siendo S el conjunto de puntos en los que están definidas.

Teorema 2 (Teorema del valor medio o de Lagrange.). *Si $f \in C[a, b]$ y es derivable en (a, b) , entonces existe un punto $c \in (a, b)$ tal que*

$$f'(c) = \frac{f(b) - f(a)}{b - a}.$$

Geoméricamente hablando, Figura 3, el teorema del valor medio dice que hay al menos un número $c \in (a, b)$ tal que la pendiente de la recta tangente a la curva $y = f(x)$ en el punto $(c, f(c))$ es igual a la pendiente de la recta secante que pasa por los puntos $(a, f(a))$ y $(b, f(b))$.

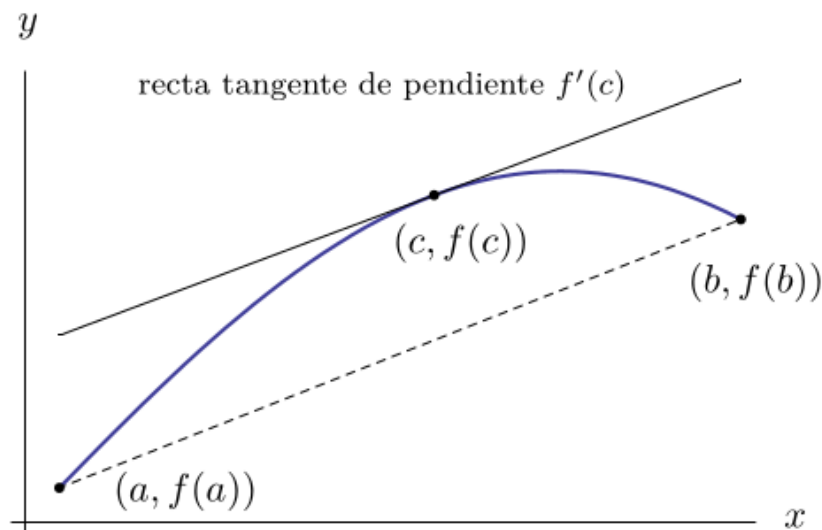


Figura 3: Teorema del valor medio o de Lagrange

Teorema 3 (Teorema de los valores extremos.). Si $f \in C[a, b]$, entonces existen c_1 y c_2 en (a, b) tales que $f(c_1) \leq f(x) \leq f(c_2)$ para todo $x \in [a, b]$. Si además, f es derivable en (a, b) , entonces los puntos c_1 y c_2 están en los extremos de $[a, b]$ o bien son puntos críticos.

3.1.4. Integración

Teorema 4 (Primer teorema fundamental o regla de Barrow.). Si $f \in C[a, b]$ y F es una primitiva cualquiera de f en $[a, b]$ (es decir, $F'(x) = f(x)$), entonces

$$\int_a^b f(x) dx = F(b) - F(a).$$

Teorema 5 (Segundo teorema fundamental.). Si $f \in C[a, b]$ y $x \in (a, b)$, entonces

$$\frac{d}{dx} \int_a^x f(t) dt = f(x).$$

Teorema 6 (Teorema del valor medio para integrales.). Si $f \in C[a, b]$, g es integrable en $[a, b]$ y $g(x)$ no cambia de signo en $[a, b]$, entonces existe un punto $c \in (a, b)$ tal que

$$\int_a^b f(x)g(x) dx = f(c) \int_a^b g(x) dx.$$

Nota 2. Cuando $g(x) = 1$, véase la Figura 4, este resultado es el habitual teorema del valor medio para integrales y proporciona el valor medio de la función f en el intervalo $[a, b]$, que está dado por

$$f(c) = \frac{1}{b-a} \int_a^b f(x) dx.$$

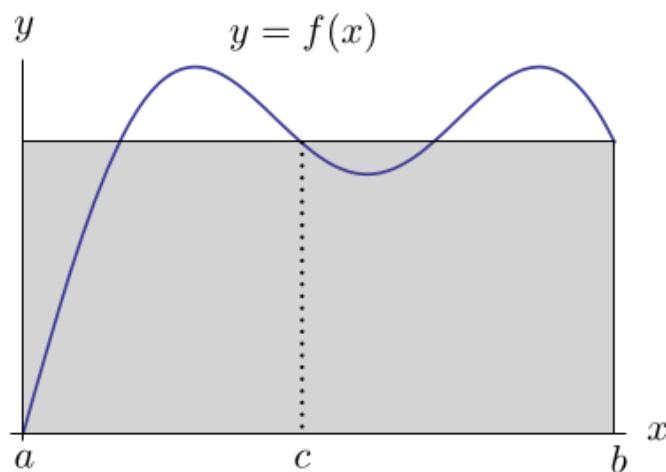


Figura 4: Teorema del valor medio para integrales

3.1.5. Polinomios de Taylor

Teorema 7 (Teorema de Taylor.). Supongamos que $f \in C^{(n)}[a, b]$ y que $f^{(n+1)}$ existe en $[a, b]$. Sea x_0 un punto en $[a, b]$. Entonces, para cada x en $[a, b]$, existe un punto $\xi(x)$ entre x_0 y x tal que

$$f(x) = P_n(x) + R_n(x),$$

donde

$$P_n(x) = f(x_0) + f'(x_0)h + \frac{f''(x_0)}{2!}h^2 + \cdots + \frac{f^{(n)}(x_0)}{n!}h^n = \sum_{k=0}^n \frac{f^{(k)}(x_0)}{k!}h^k, \quad (1)$$

$$R_n(x) = \frac{f^{(n+1)}(\xi(x))}{(n+1)!}h^{n+1}, \quad y \quad h = x - x_0. \quad (2)$$

El polinomio $P_n(x)$ se llama **n-ésimo polinomio de Taylor** de f alrededor de x_0 (véase la Figura 5). $R_n(x)$ se llama **error de truncamiento** (o *resto de Taylor*) asociado a $P_n(x)$. Como el punto $\xi(x)$ en el error de truncamiento $R_n(x)$ depende del punto x en el que se evalúa el polinomio $P_n(x)$, podemos verlo como una función de la variable x .

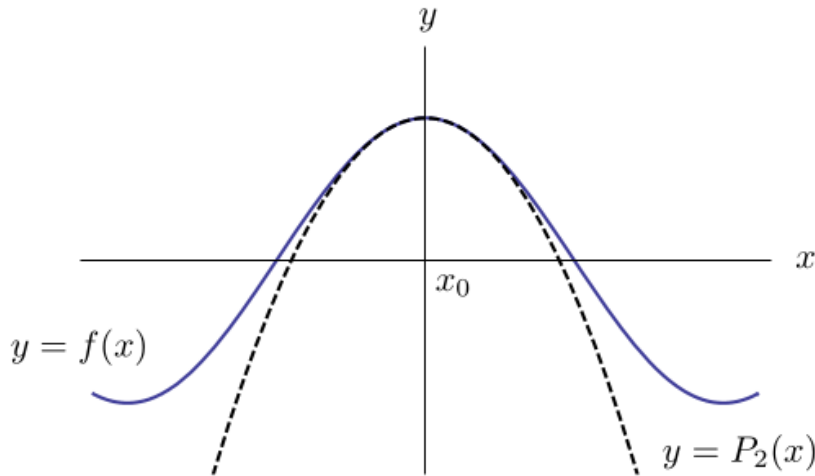


Figura 5: Gráficas de $y = f(x)$ y de su polinomio de Taylor $y = P_2(x)$ alrededor de x_0 .

Nota 3. La serie infinita que resulta al tomar límite en la expresión de $P_n(x)$ cuando $n \rightarrow \infty$ se llama **serie de Taylor** de f alrededor de x_0 . Cuando $x_0 = 0$, el polinomio de Taylor se suele denominar **polinomio de Maclaurin**, y la serie de Taylor se llama **serie de Maclaurin**.

La denominación error de truncamiento en el teorema de Taylor se refiere al error que se comete al usar una suma truncada al aproximar la suma de una serie infinita.

3.1.6. Teoremas adicionales

A continuación enunciaremos algunas de las definiciones y teoremas básicos que utilizaremos a lo largo de estas notas.

Definición 1. f es de clase C^1 en el intervalo $[a; b]$ si f' es continua en $[a; b]$.

Definición 2. f es de clase C^n en el intervalo $[a; b]$ si $f^{(n)}$ es continua en $[a; b]$.

Definición 3. f es de clase C^∞ en el intervalo I si f es infinitas veces derivable y continua en I .

Teorema 8. (Teorema de los valores intermedios). Sea f continua en el intervalo $[a; b]$. Si $k \in \mathbb{R}$ es un número comprendido entre $f(a)$ y $f(b)$, entonces existe al menos un punto ξ perteneciente al intervalo $(a; b)$ tal que $f(\xi) = k$.

Teorema 9 (Bolzano). Si f es continua en el intervalo $[a; b]$ y $f(a) \cdot f(b) < 0$, entonces existe un ξ perteneciente al $(a; b)$ tal que $f(\xi) = 0$.

Teorema 10 (Teorema de acotabilidad). Si $f : [a; b] \mapsto \mathbb{R}$ es continua en $[a; b]$, entonces f está acotada en $[a; b]$.

Teorema 11 (Teorema de Weierstrass). Si $f : [a; b] \mapsto \mathbb{R}$ es continua en $[a; b]$, entonces f tiene un máximo global y un mínimo global en $[a; b]$.

Teorema 12 (Teorema Generalizado de Rolle). Si f continua en $[a; b]$, y existen las derivadas $f'(x), f''(x), \dots, f^{(n)}(x)$ en $(a; b)$ y $f(x_0) = f(x_1) = \dots = f(x_n) = 0$ (con $x_0, x_1, \dots, x_n \in [a; b]$) entonces existe ξ perteneciente al $(a; b)$ tal que $f^{(n)}(\xi) = 0$.

Teorema 13 (Teorema de Lagrange). Si f continua en $[a; b]$ y derivable en $(a; b)$ entonces existe ξ perteneciente al $(a; b)$ tal que $f(b) - f(a) = f'(\xi)(b - a)$.

Teorema 14 (Teorema del valor medio ponderado). Sea f continua en $[a, b]$ y g una función integrable Riemann en $[a, b]$. Si g no cambia de signo en $[a, b]$, entonces existe un número $c \in (a, b)$ tal que:

$$\int_a^b f(x)g(x)dx = f(c) \int_a^b g(x)dx$$

3.2. Álgebra Lineal

3.2.1. Matrices

Una **matriz** es un arreglo multidimensional de escalares, llamados *elementos*, ordenados en filas y columnas. Una matriz de m filas y n columnas, o *matriz (de orden) $m \times n$* , es un conjunto de $m \cdot n$ elementos a_{ij} , con $i = 1, 2, \dots, m$ y $j = 1, 2, \dots, n$, que se representa de la siguiente forma:

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}$$

Se puede abreviar la representación de la matriz anterior de la forma $A = (a_{ij})$ con $i = 1, 2, \dots, m$; $j = 1, 2, \dots, n$.

Hay una relación directa entre matrices y vectores puesto que podemos pensar una matriz como una composición de vectores fila o de vectores columna. Además, un vector es un caso especial de matriz: un *vector fila* es una matriz con una sola fila y varias columnas, y un *vector columna* es una matriz con varias filas y una sola columna.

3.2.2. Operaciones con matrices

- Si $A = (a_{ij})$ y $B = (b_{ij})$ son dos matrices que tienen el mismo orden, $m \times n$, decimos que A y B son **iguales** si $a_{ij} = b_{ij}$ para todo $i = 1, \dots, m$ y $j = 1, \dots, n$.
- Si $A = (a_{ij})$ y $B = (b_{ij})$ son dos matrices que tienen el mismo orden $m \times n$, la **suma** de A y B es una matriz $C = (c_{ij})$ del mismo orden con $c_{ij} = a_{ij} + b_{ij}$ para todo $i = 1, \dots, m$ y $j = 1, \dots, n$.
- Si $A = (a_{ij})$ es una matriz de orden $m \times n$, la **multiplicación de A por un escalar λ** es una matriz $C = (c_{ij})$ del mismo orden $m \times n$ con $c_{ij} = \lambda a_{ij}$ para todo $i = 1, \dots, m$ y $j = 1, \dots, n$.
- Si $A = (a_{ij})$ es una matriz de orden $m \times n$, la **matriz traspuesta** de A es la matriz que resulta de intercambiar sus filas por sus columnas, se denota por A^T y es de orden $n \times m$.
- Si $A = (a_{ij})$ es una matriz de orden $m \times p$ y $B = (b_{ij})$ es una matriz de orden $p \times n$, el **producto de A por B** es una matriz $C = (c_{ij})$ de orden $m \times n$ con

$$c_{ij} = \sum_{k=1}^p a_{ik} b_{kj}, \quad \text{para todo } i = 1, \dots, m \text{ y } j = 1, \dots, n.$$

Obsérvese que el producto de dos matrices solo está definido si el número de columnas de la primera matriz coincide con el número de filas de la segunda.

3.2.3. Matrices especiales

- Una matriz $A = (a_{ij})$ es **cuadrada** si tiene el mismo número de filas que de columnas, y de orden n si tiene n filas y n columnas. Se llama **diagonal principal** al conjunto de elementos $a_{11}, a_{22}, \dots, a_{nn}$.
- Una **matriz diagonal** es una matriz cuadrada que tiene algún elemento distinto de cero en la diagonal principal y ceros en el resto de elementos.
- Una matriz cuadrada con ceros en todos los elementos por encima (debajo) de la diagonal principal se llama **matriz triangular inferior (superior)**.
- Una matriz diagonal con unos en la diagonal principal se denomina **matriz identidad** y se denota por I . Es la única matriz cuadrada tal que $AI = IA = A$ para cualquier matriz cuadrada A .
- Una **matriz simétrica** es una matriz cuadrada A tal que $A = A^T$.
- La **matriz cero** es una matriz con todos sus elementos iguales a cero.
- Decimos que una matriz cuadrada A es **invertible** (o *regular* o *no singular*) si existe una matriz cuadrada B tal que $AB = BA = I$. Se dice entonces que B es la **matriz inversa** de A y se denota por A^{-1} . (Una matriz que no es invertible se dice *singular*.)
- Si una matriz A es invertible, su inversa también lo es y $(A^{-1})^{-1} = A$.
- Si A y B son dos matrices invertibles, su producto también lo es y $(AB)^{-1} = B^{-1}A^{-1}$.

3.2.4. Determinante de una matriz

El **determinante** de una matriz solo está definido para matrices cuadradas y su valor es un escalar. El determinante de una matriz A cuadrada de orden n se denota por $|A|$ o $\det(A)$, y se define como

$$\det(A) = \sum_j (-1)^{i+j} a_{ij} \cdot \det(A_{ij}),$$

donde la suma se toma para todas las $n!$ permutaciones de grado n y s es el número de intercambios necesarios para poner el segundo subíndice en el orden $1, 2, \dots, n$.

Algunas propiedades de los determinantes son:

- $\det(A^T) = \det(A)$
- $\det(AB) = \det(A) \det(B)$
- $\det(A^{-1}) = \frac{1}{\det(A)}$
- Si dos filas o dos columnas de una matriz coinciden, el determinante de esta matriz es cero.
- Cuando se intercambian dos filas o dos columnas de una matriz, su determinante cambia de signo.
- El determinante de una matriz diagonal es el producto de los elementos de la diagonal.
- Si denotamos por A_{ij} la matriz de orden $(n-1)$ que se obtiene de eliminar la fila i y la columna j de la matriz A , llamamos **menor complementario** asociado al elemento a_{ij} de la matriz A al $\det(A_{ij})$.
- Se llama **k -ésimo menor principal** de la matriz A al determinante de la submatriz principal de orden k .
- Definimos el **cofactor** del elemento a_{ij} de la matriz A por $\Delta_{ij} = (-1)^{i+j} \det(A_{ij})$.
- Si A es una matriz invertible de orden n , entonces

$$A^{-1} = \frac{1}{\det(A)} C,$$

donde C es la matriz de elementos Δ_{ij} para todo $i, j = 1, 2, \dots, n$. Obsérvese entonces que una matriz cuadrada es invertible si y sólo si su determinante es distinto de cero.

3.2.5. Valores propios y vectores propios

- Si A es una matriz cuadrada de orden n , un número λ es un **valor propio** de A si existe un vector no nulo v tal que $Av = \lambda v$. Al vector v se le llama **vector propio** asociado al valor propio λ .
- El valor propio λ es solución de la **ecuación característica**

$$\det(A - \lambda I) = 0,$$

donde $\det(A - \lambda I)$ se llama **polinomio característico**. Este polinomio es de grado n en λ y tiene n valores propios (no necesariamente distintos).

3.2.6. Normas vectoriales y normas matriciales

Para medir la **longitud** de los vectores y el **tamaño** de las matrices se suele utilizar el concepto de **norma**, que es una función que toma valores reales. Un ejemplo simple en el espacio euclidiano tridimensional es un vector $v = (v_1, v_2, v_3)$, donde v_1, v_2 y v_3 son las distancias a lo largo de los ejes x, y, z respectivamente.

La **longitud del vector** v (es decir, la distancia del punto $(0, 0, 0)$ al punto (v_1, v_2, v_3)) se calcula como

$$\|v\| = \sqrt{v_1^2 + v_2^2 + v_3^2},$$

donde la notación $\|v\|$ indica que esta longitud se refiere a la *norma euclidiana* del vector v . De forma similar, para un vector v de dimensión n , $v = (v_1, v_2, \dots, v_n)$, la norma euclidiana se calcula como

$$\|v\| = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}.$$

Este concepto puede extenderse a una matriz $m \times n$, $A = (a_{ij})$, de la siguiente manera:

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2},$$

que recibe el nombre de **norma de Frobenius**.

Hay otras alternativas a las normas euclidiana y de Frobenius. Dos normas usuales son la **norma 1** y la **norma infinito**:

- La **norma 1** de un vector $v = (v_1, v_2, \dots, v_n)$ se define como $\|v\|_1 = \sum_{i=1}^n |v_i|$. De forma similar, la norma 1 de una matriz $m \times n$, $A = (a_{ij})$, se define como

$$\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^m |a_{ij}|.$$

- La **norma infinito** de un vector $v = (v_1, v_2, \dots, v_n)$ se define como $\|v\|_\infty = \max_i |v_i|$. La norma infinito de una matriz $m \times n$, $A = (a_{ij})$, se define como

$$\|A\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|.$$

- Todas las normas son equivalentes en un espacio vectorial de dimensión finita.

4. Operaciones de punto flotante

4.1. Sistemas decimal y binario

El sistema numérico que se utiliza frecuentemente es el sistema decimal, en la que la base de expresión es el 10. Sin embargo las computadoras utilizan el sistema binario, sistema de base 2, es decir solamente $\{0, 1\}$.

Proposición 1. Para cualquier número natural N , existen $a_0, a_1, a_2, \dots, a_K$, con $a_i \in \mathbb{R}$ tales que

$$N = a_K \times 2^K + a_{K-1} \times 2^{K-1} + a_{K-2} \times 2^{K-2} + \dots + a_1 \times 2 + a_0 \times 2^0 \quad (3)$$

Para ver lo anterior lo que tenemos que hacer es calcular $\frac{N}{2}$, es decir, $\frac{N}{2} = P_0 + \frac{a_0}{2}$, donde $P_0 = a_K \times 2^{K-1} + a_{K-1} \times 2^{K-2} + a_{K-2} \times 2^{K-3} + \dots + a_1 \times 2^0$, es decir a_0 es el resto de dividir N entre 2. Ahora hagamos lo mismo para P_0 :

$$\frac{P_0}{2} = a_K \times 2^{K-2} + a_{K-1} \times 2^{K-3} + a_{K-2} \times 2^{K-4} + \dots + a_2 \times 2^0 + \frac{a_1}{2},$$

por lo tanto

$$\frac{P_0}{2} = P_1 + \frac{a_1}{2},$$

donde

$$P_1 = a_K \times 2^{K-2} + a_{K-1} \times 2^{K-3} + a_{K-2} \times 2^{K-4} + \dots + a_2 \times 2^0,$$

es decir P_1 es el resto de dividir P_0 entre 2. Siguiendo este procedimiento de manera análoga hasta que encontremos un valor K tal que $P_K = 0$. Por lo tanto tenemos el siguiente algoritmo:

Algoritmo 1. Para un valor N natural, los términos a_k en la ecuación 3 se encuentran

$$\begin{aligned} N &= 2P_0 + a_0, \\ P_0 &= 2P_1 + a_1, \\ &\vdots \\ P_{K-2} &= 2P_{K-1} + a_{K-1}, \\ P_{K-1} &= 2P_K + a_K, \\ P_K &= 0. \end{aligned} \quad (4)$$

Ejemplo 1. Convertir 24563

$$\begin{aligned} 24563 &= 12281 \times 2 + 1, & a_0 &= 1 \\ 12281 &= 6140 \times 2 + 1, & a_1 &= 1 \\ 6140 &= 3070 \times 2 + 0, & a_2 &= 0 \\ 3070 &= 1535 \times 2 + 0, & a_3 &= 0 \\ 1535 &= 767 \times 2 + 1, & a_4 &= 1 \\ 767 &= 383 \times 2 + 1, & a_5 &= 1 \\ 383 &= 191 \times 2 + 1, & a_6 &= 1 \\ 191 &= 95 \times 2 + 1, & a_7 &= 1 \\ 95 &= 47 \times 2 + 1, & a_8 &= 1 \\ 47 &= 23 \times 2 + 1, & a_9 &= 1 \\ 23 &= 11 \times 2 + 1, & a_{10} &= 1 \\ 11 &= 5 \times 2 + 1, & a_{11} &= 1 \\ 5 &= 2 \times 2 + 1, & a_{12} &= 1 \\ 2 &= 1 \times 2 + 0, & a_{13} &= 0 \\ 1 &= 0 \times 2 + 1, & a_{14} &= 1 \end{aligned}$$

Por lo tanto el número binario es: $(24563)_{10} = (101111111110011)_2$.

Proposición 2. Sea $Q \in \mathbb{R}$, tal que $0 < Q < 1$, entonces existen términos $b_1, b_2, \dots, b_k$ tales que $Q = 0.b_1b_2b_3 \dots b_k$, y por tanto

$$Q = b_1 \times 2^{-1} + b_2 \times 2^{-2} + b_3 \times 2^{-3} + \dots + b_k \times 2^{-k} + \dots \quad (5)$$

Si multiplicamos Q por 2, se tiene que

$$2Q = b_1 + b_2 \times 2^{-1} + b_3 \times 2^{-2} + b_4 \times 2^{-3} + \dots + b_k \times 2^{-k+1} + \dots$$

Si $F_1 = \text{frac}(2Q)$, con $\text{frac}(x)$ la parte fraccionaria de x , y $b_1 = \lfloor 2Q \rfloor$, donde $\lfloor x \rfloor$ es la parte entera de x , entonces

$$F_1 = b_2 \times 2^{-1} + b_3 \times 2^{-2} + b_4 \times 2^{-3} + \dots + b_k \times 2^{-k+1} + \dots,$$

de donde

$$2F_1 = b_2 \times 2^0 + b_3 \times 2^{-1} + b_4 \times 2^{-2} + \dots + b_k \times 2^{-k+2} + \dots = b_2 + F_2,$$

donde $F_2 = \text{frac}(2F_1)$, y $b_2 = \lfloor 2F_1 \rfloor$. Procediendo de manera análoga para el resto de los términos se tienen las sucesiones $\{b_k\}$ y $\{F_k\}$, dadas por $b_k = \lfloor 2F_{k-1} \rfloor$ y $F_k = \text{frac}(2F_{k-1})$, con $b_1 = \lfloor 2Q \rfloor$ y $F_1 = \text{frac}(2Q)$. Por lo tanto se tiene la representación binaria de Q dada por

$$Q = \sum_{i=1}^{\infty} b_i 2^{-i} \quad (6)$$

Ejemplo 2. Convertir el número 3.5786. Sea $Q = 0.5786$, entonces

$$\begin{aligned} 2Q &= 1.1572, b_1 = \lfloor 1.1572 \rfloor = 1, F_1 = \text{frac}(1.1572) = 0.1572 \\ 2F_1 &= 0.3144, b_2 = \lfloor 0.3144 \rfloor = 0, F_2 = \text{frac}(0.3144) = 0.3144 \\ 2F_2 &= 0.6288, b_3 = \lfloor 0.6288 \rfloor = 0, F_3 = \text{frac}(0.6288) = 0.6288 \\ 2F_3 &= 1.2576, b_4 = \lfloor 1.2576 \rfloor = 1, F_4 = \text{frac}(1.2576) = 0.2576 \\ 2F_4 &= 0.5152, b_5 = \lfloor 0.5152 \rfloor = 0, F_5 = \text{frac}(0.5152) = 0.5152 \\ 2F_5 &= 1.0304, b_6 = \lfloor 1.0304 \rfloor = 1, F_6 = \text{frac}(1.0304) = 0.0304 \\ 2F_6 &= 0.0608, b_7 = \lfloor 0.0608 \rfloor = 0, F_7 = \text{frac}(0.0608) = 0.0608 \\ 2F_7 &= 0.1216, b_8 = \lfloor 0.1216 \rfloor = 0, F_8 = \text{frac}(0.1216) = 0.1216 \end{aligned}$$

De lo anterior se tiene que:

$$0.5786 = (0.10010100 \dots)_2$$

Por lo tanto:

$$3.5786 = (11.10010100 \dots)_2.$$

Ejercicio 1. Convertir los siguientes números de base 10 a base 2.

1. 324
2. 27
3. 1423
4. 235.25
5. 41.596

4.2. Números en punto flotante

Definición 4. Los números en punto flotante son número reales de la forma

$$\pm\alpha \times \beta^e, \quad (7)$$

donde α tiene un número de dígitos limitados, β es la base y e es el exponente que modifica la posición del punto decimal.

Definición 5. Un número real x tiene una representación punto flotante normalizada si

$$\pm\alpha \times \beta^e, \quad (8)$$

con $\frac{1}{\beta} < |\alpha| < 1$

Nota 4. En este caso x se puede escribir en la forma

$$x = \pm 0, d_1 d_2 \cdots d_k \times \beta^e, \quad (9)$$

donde si $x \neq 0, d_1 \neq 0$, y además $0 \leq d_i < \beta$, para $i = 1, 2, 3, \dots, k$ y $L \leq e \leq U$.

Definición 6. El conjunto de los números en punto flotante se le llama **conjunto de números máquina**.

Nota 5. El conjunto de números máquina es finito. Para ver esto consideremos que si x es de la forma

$$\pm 0, d_1 d_2 \cdots d_t \times \beta^e, \quad (10)$$

dado que $d_1 \neq 0$ y $0 \leq d_i < \beta$ entonces d_1 puede tomar $\beta - 1$ valores distintos, mientras que para d_i hay β posibilidades. Por lo tanto se tienen $(\beta - 1) \beta \beta \cdots \beta = (\beta - 1) \beta^t$. El número de exponentes posibles son $U - L + 1$, en total hay $(\beta - 1) \beta^t (U - L + 1)$ números máquina positivos, por lo tanto, considerando positivos y negativos hay $2(\beta - 1) \beta^t (U - L + 1)$ números máquina, considerando que el cero también es un número de máquina hay en realidad $2(\beta - 1) \beta^t (U - L + 1) + 1$. Es decir, cualquier número real puede ser representado por uno de los $2(\beta - 1) \beta^t (U - L + 1) + 1$ números de máquina.

Ejemplo 3. Recordemos la expresión (10), consideremos $\beta = 2$, $t = 3$, $L = -2$ y $U = 2$. Entonces $x = \pm d_1 d_2 d_3 \times (2)^e$, con $-2 \leq e \leq 2$ y $0 \leq d_1, d_2 < 2$. Por lo tanto se tiene que $d_1, d_2, d_3 = 1$; $e = \{-2, -1, 0, 1, 2\}$. Entonces $d_1 d_2 d_3 = \{100, 101, 110, 111\} = \{\frac{1}{2}, \frac{5}{8}, \frac{3}{4}, \frac{7}{8}\}$

-2	-1	0	1	2
0.111×2^{-2}	0.111×2^{-1}	0.111×2^0	0.111×2^1	0.111×2^2
0.110×2^{-2}	0.110×2^{-1}	0.110×2^0	0.110×2^1	0.110×2^2
0.101×2^{-2}	0.101×2^{-1}	0.101×2^0	0.101×2^1	0.101×2^2
0.100×2^{-2}	0.100×2^{-1}	0.100×2^0	0.100×2^1	0.100×2^2

sustituyendo y resolviendo las operaciones

-2	-1	0	1	2
$\frac{7}{8} \times 2^{-2} = \frac{7}{32}$	$\frac{7}{8} \times 2^{-1} = \frac{7}{16}$	$\frac{7}{8} \times 2^0 = \frac{7}{8}$	$\frac{7}{8} \times 2^1 = \frac{7}{4}$	$\frac{7}{8} \times 2^2 = \frac{7}{2}$
$\frac{3}{4} \times 2^{-2} = \frac{3}{16}$	$\frac{3}{4} \times 2^{-1} = \frac{3}{8}$	$\frac{3}{4} \times 2^0 = \frac{3}{4}$	$\frac{3}{4} \times 2^1 = \frac{3}{2}$	$\frac{3}{4} \times 2^2 = 3$
$\frac{5}{8} \times 2^{-2} = \frac{5}{32}$	$\frac{5}{8} \times 2^{-1} = \frac{5}{16}$	$\frac{5}{8} \times 2^0 = \frac{5}{8}$	$\frac{5}{8} \times 2^1 = \frac{5}{4}$	$\frac{5}{8} \times 2^2 = \frac{5}{2}$
$\frac{1}{2} \times 2^{-2} = \frac{1}{8}$	$\frac{1}{2} \times 2^{-1} = \frac{1}{4}$	$\frac{1}{2} \times 2^0 = \frac{1}{2}$	$\frac{1}{2} \times 2^1 = 1$	$\frac{1}{2} \times 2^2 = 2$

Por tanto el número total de números máquina son 41.

Ejercicio 2. 1. Describir todos los números máquina para $\beta = 2$, $t = 4$, $L = -3$ y $U = 3$.

2. Escribir el número 732.5051 en notación de punto flotante. Respuesta 0.7325051×10^{-3}

3. Escribir el número 0.006521 en notación de punto flotante. Respuesta 0.06521×10^{-2}

4. $(101.01)_2$. Respuesta 0.10101×2^3

5. $(0.00101111)_2$. Respuesta 0.101111×2^{-2}

Nota 6. $(0.1)_2 = 1 \times 2^{-1} = 0.5$

4.3. Representación

En una computadora los números se expresan como se ha descrito, pero con restricciones sobre q y m dadas por la longitud de la palabra. Supongamos que la longitud de la palabra es de 32 bits, los cuales se distribuyen de la siguiente manera: los dos primeros se reservan para los signos: es 0 si es positivo y 1 si es negativo; los siguientes 7 espacios para el exponente, y los restantes para la mantisa. Considerando que cualquier número puede normalizarse, recordar $x = \pm q \times 2^m$, con $\frac{1}{2} \leq q < 1$, se puede asumir que el primer bit en q es 1 y por tanto no se requiere almacenar,.

Ejemplo 4. Representar y almacenar en punto flotante normalizado el número -0.125 . A saber $(-0.125) = (-0.001)_2 = (-0.1)_2 \times 2^{-2}$, además $2 = (10)_2$; por lo tanto su representación es: 1, 1|, 0, 0, 0, 0, 0, 1, 0| 0, 0, 0, 0, 0, 0, 0, ... 0.

Ejemplo 5. Represente y almacene el número 117.125. Respuesta $117 = (1110101)_2$ y $0.125 = (0.001)_2$, por tanto $117.125 = (1110101.001)_2 = (0.1110101001)_2 \times 2^7$ y $y = (111)_2$, por tanto se almacena: 0, 0|0000111|1110101 ... 0

Nota 7. $|m|$ no requiere más de 7 bits, es decir $|m| \leq (1111111)_2 = 2^7 - 1 = 127$, por tanto el exponente de 7 dígitos binarios proporciona un intervalo de 0 a 127, para números pequeños se toma el exponente en el intervalo $[-63, 64]$. Además q requiere de a lo más 24 bits, por tanto la máquina de 32 bits tienen una precisión limitada entre 7 y 8 dígitos decimales, ya que el bit menos significativo en la mantisa representa unidades del orden $2^{-24} \approx 10^{-7}$. Esto quiere decir que números expresados por más de siete dígitos decimales serán aproximados cuando se dan como datos de entrada o como resultados de operaciones.

4.4. Errores

A la hora de realizar un cálculo es importante asegurarse de que los números que intervienen en el cálculo se pueden utilizar con confianza. Para ello, se introduce el concepto de **cifras significativas**, que designa formalmente la notación de un valor numérico, y se usa en aquellos que guían visualmente la precisión.

Los **errores de truncamiento** se producen cuando utilizamos una aproximación en lugar de un procedimiento matemático exacto. Para conocer las características de estos errores se suelen considerar los polinomios de Taylor. Cuando se aproxima un proceso continuo por uno discreto, para errores provocados por un tamaño de paso finito h , resulta a menudo útil describir la dependencia del error e con h cuando h tiende a cero.

Decimos que una función $f(h)$ es una $\mathcal{O}$ grande de h^n si $|f(h)| \leq ch^n$ para alguna constante c , cuando h es cercano a cero; se escribe

$$f(h) = \mathcal{O}(h^n) \quad (13)$$

Si un método tiene un término de error que es $\mathcal{O}(h^n)$, se suele decir que es un **método de orden n** . Por ejemplo, si utilizamos un polinomio de Taylor para aproximar la función g en $x = a + h$, tenemos:

$$g(x) = g(a + h) = g(a) + hg'(a) + \frac{h^2}{2!}g''(a) + \frac{h^3}{3!}g^{(3)}(\xi), \quad \text{para algún } \xi \in [a, a + h].$$

Suponiendo que g es suficientemente derivable, la aproximación anterior es $\mathcal{O}(h^3)$, puesto que el error

$$\frac{h^3}{3!}g^{(3)}(\xi), \quad \text{satisface} \quad \left| \frac{h^3}{3!}g^{(3)}(\xi) \right| \leq \frac{M}{3!}|h^3|,$$

donde M es el máximo de $g^{(3)}(x)$ para $x \in [a, a + h]$.

Las diferencias (errores) son múltiples y de diversa naturaleza, aunque pueden separarse en dos grupos genéricos:

- **Los errores que provienen del modelado teórico** (o abstracción matemática) del fenómeno real; estos errores se denominan *Errores del modelo o inherentes*. Los errores inherentes son producto de factores intrínsecos a la naturaleza, al ambiente y las personas mismas. Los errores inherentes son imposibles de remediar aunque pueden minimizarse; en consecuencia, no pueden cuantificarse.

Se distinguen dos tipos de errores inherentes: **Las incertidumbres** hacen referencia a las dimensiones físicas que nunca podrán ser medidas en forma exacta debido a la naturaleza de la materia y a las imperfecciones de los instrumentos de medición. **Las verdaderas equivocaciones** son las situaciones que se producen en la lectura de instrumentos de medición o en el traslado de información y que son inadvertidas a las personas; un claro ejemplo de estas situaciones es la denominada *ceguera de taller*.

- **Los errores del método** son producto de la limitante en la representación y manipulación de cantidades numéricas utilizadas en los cálculos necesarios en el desarrollo del modelo matemático. Es de destacar que los dispositivos de cálculo (tales como calculadoras y computadoras) utilizan y manipulan cantidades en forma imprecisa.

Existen dos grandes tipos de errores del método: *El truncamiento* se provoca ante la imposibilidad de manipular, por parte de un instrumento de cómputo, una cantidad infinita de términos o cifras. Los términos o cifras omitidas (que son infinitas en número) introducen un error en los resultados calculados. *El redondeo* se produce por el mismo motivo que el truncamiento pero, a diferencia de éste, las cifras omitidas sí son consideradas en la cifra resultante.

En general, si se incrementa el número de cifras significativas en el ordenador, se minimizan los errores de redondeo, y los errores de truncamiento disminuyen a medida que los errores de redondeo se incrementan. Por lo tanto, para disminuir uno de los dos sumandos del error total debemos incrementar el otro. Como el error total no se puede calcular en la mayoría de los casos, se suelen utilizar otras medidas para estimar la exactitud de un método numérico, que suelen depender del método específico. En algunos métodos el error numérico se puede acotar, mientras que en otros se determina una estimación del *orden de magnitud del error*. Cuando se buscan las soluciones numéricas de un problema real los resultados que se obtienen por lo general no son exactos.

4.5. Cuantificación de errores

Los errores se cuantifican de dos formas diferentes:

1. **Error absoluto.** El error absoluto es la diferencia absoluta entre un valor real y un aproximado. Está dado por la siguiente fórmula:

$$E = |V_{real} - V_{aprox}|$$

El error absoluto recibe este nombre ya que posee las mismas dimensiones que la variable bajo estudio.

2. **Error relativo.** Corresponde a la expresión en porcentaje de un error absoluto; en consecuencia, este error es adimensional.

$$e = \left| \frac{V_{real} - V_{aprox}}{V_{real}} \right| \cdot 100 \%$$

La diferencia entre la preferencia en el uso de los dos tipos de error consiste precisamente en la presencia de las dimensiones físicas. Debido a las unidades de medición utilizadas, el manejo y la percepción del error absoluto suele ser engañoso o difícil de comprender rápidamente. Sin embargo, el manejo de porcentajes (o valores relativos) resulta más natural y sencillo de comprender. Sin embargo, el uso de estos dos tipos de errores está sujeto siempre al objetivo de las actividades desarrolladas.

Las expresiones que definen a los errores absoluto y relativo requieren del conocimiento de la variable V_{real} que representa un valor ideal que no posee error alguno. Como podrá suponerse, en la realidad resulta imposible determinar este valor. Una práctica común en los análisis elementales sobre errores es considerar como un valor real a los resultados arrojados por la medición experimental de los fenómenos y a los valores aproximados como los proporcionados por los modelos matemáticos (o viceversa). En realidad, ambos valores son valores aproximados. Para lograr un resultado coherente, en la práctica debe sustituirse al valor real por un valor que se considere posee un error menor.

En el caso del análisis numérico, dado que los resultados se obtienen a partir de procesos iterativos que se mejoran los inicialmente obtenidos, debe partirse del supuesto que el último valor obtenido posee un nivel menor de error que el valor previo. Dado lo anterior, los errores absoluto y relativo se calcularán de la siguiente forma:

Error absoluto:

$$E = |V_i - V_{i-1}|$$

Error relativo:

$$e = \left| \frac{V_i - V_{i-1}}{V_i} \right| \cdot 100 \%$$

En ambas ecuaciones, V_i es el valor de la última iteración y V_{i-1} es el valor de la iteración anterior $i-1$.

- **Error absoluto:** Se define como la diferencia entre el valor real (experimental o exacto) y el valor aproximado (teórico o calculado):

$$E_a = |x_{real} - x_{aproximado}|$$

- **Error relativo:** Es la relación entre el error absoluto y el valor real:

$$E_r = \frac{E_a}{|x_{\text{real}}|}$$

- **Error porcentual:** Es el error relativo expresado en porcentaje:

$$E_p = E_r \cdot 100 = \frac{E_a}{|x_{\text{real}}|} \cdot 100$$

4.6. Errores en punto flotante

Un número real $x \in \mathbb{R}$, cuando no pertenece a F , se representa mediante una aproximación flotante $fl(x) \in F$, de modo que:

$$x = fl(x)(1 + \varepsilon), \quad |\varepsilon| \leq \epsilon_{\text{mach}}.$$

Este ε se llama **error relativo** y ϵ_{mach} es la **precisión de la máquina**. En general se tiene que:

$$\epsilon_{\text{mach}} = \frac{1}{2}\beta^{1-t}.$$

Nota 8. Los ordenadores almacenan los números reales en forma binaria (base 2). Cada dígito binario (0 ó 1) se llama **bit** (por digital binary). La memoria de los ordenadores está organizada en **bytes**, siendo un byte = 8 bits. En el estándar IEEE, los ordenadores representan números reales en precisión simple (32 bits) y en precisión doble (64 bits). Este estándar también define los códigos para representar valores especiales como NaN (Not a Number), infinitos, y ceros con signo. Para representar un número real, se utiliza la forma normalizada:

$$x = (-1)^s \cdot (1 + f) \cdot 2^e,$$

donde: s : es el bit de signo (0 para positivo, 1 para negativo), f : es la fracción, e : es el exponente con sesgo.

Por ejemplo, el número 0.15625 en binario es 0.00101, que se escribe como $1.01 \cdot 2^{-3}$ y se almacena como: signo = 0, exponente = 124 (con sesgo de 127), y mantisa = 010000... En este formato, la precisión depende de la cantidad de bits reservados a la fracción f . En doble precisión (64 bits), se reservan 52 bits para la fracción, 11 para el exponente y 1 para el signo.

4.7. Aproximación numérica y errores

Una *aproximación* es un valor cercano a uno considerado como real o verdadero. Esta cercanía, o diferencia, se conoce como *error*. Normalmente, la consideración de la validez de una aproximación depende de la cota de error que el experimentador considere pertinente en función del contexto del fenómeno bajo estudio. Esto implica que también debe considerarse que una magnitud debe ser un valor real, que en el ámbito de la Ingeniería pocas veces se conoce, lo que obliga a adoptar convenciones. En Ingeniería, se denomina *exactitud* a la capacidad de un instrumento de medir un valor cercano al de la magnitud real. Exactitud implica precisión, pero no al contrario. Exactitud y precisión no son equivalentes. Exactitud es capacidad para acercarse a la magnitud real, y precisión es la capacidad de generar resultados similares. La precisión se logra cuando un instrumento para repetir mediciones exactas cuando éstas se realizan consecutivamente. De acuerdo con la definición de aproximación numérica,

la exactitud se aplica en los métodos numéricos en cuanto a la capacidad del método de generar un resultado muy cercano al valor real; se percibe la cercanía entre la exactitud y el concepto de error. Por otra parte, los métodos numéricos a través de iteraciones generan valores aproximados cada vez más exactos, es decir, estas iteraciones deberán ser precisas. Dado lo anterior, los métodos numéricos deberán tener como cualidades la exactitud y la precisión. Matemáticamente, la **convergencia** es la propiedad de algunas sucesiones y series de tender progresivamente a un límite, de tal forma, si este límite existe, se dice que la sucesión o la serie *converge*. En forma análoga, si un método numérico en su funcionamiento iterativo nos proporciona aproximaciones cada vez más cercanas al valor buscado, se dice que el método converge. La convergencia se mide a través de los errores; si el error entre dos aproximaciones sucesivas se reduce, el método converge; se debe cumplir que:

$$|x_n - x_{n-1}| \leq |x_{n-1} - x_{n-2}|$$

Es decir, la diferencia enésima ($x_n - x_{n-1}$) debe ser menor que la diferencia $(n-1)$ ésima $x_{n-1} - x_{n-2}$. Se dice que un sistema (o un proceso) es **estable** si a pequeñas variaciones en la entrada o en la excitación corresponden pequeñas variaciones en la salida o en la respuesta. La estabilidad de un método numérico tiene que ver con la manera en que los errores numéricos se propagan a lo largo del algoritmo. Cuando un método converge, lo más deseable es que en los resultados que se obtengan, los niveles de error se disminuyan en la forma más rápida posible. Sin embargo, ocurre que durante la operación del algoritmo, ya sea por el manejo de los datos numéricos o bien por la naturaleza propia del modelo matemático con el que se esté trabajando, los errores entre aproximaciones no disminuyan en forma progresiva, sino que incluso aumenten en alguna etapa del proceso para después reducirse mostrando un comportamiento aleatorio.

La **robustez** de un método numérico radica en su convergencia y su estabilidad. Pueden utilizarse métodos cuya prueba de convergencia indique la pertinencia de su uso, pero que durante su aplicación se obtengan resultados inestables que repercutan en el número de iteraciones y en consecuencia en el tiempo invertido en la solución. El ideal lo constituyen métodos que a la vez de ser convergentes resulten estables.

Nota 9. Convergencia de un método se refiere a que sea posible la obtención del valor buscado cuando el número de pasos tiende a infinito. Típicamente, la convergencia se analiza en métodos iterativos, es decir, aquellos en los que el resultado final se obtiene tras una repetición de cálculos. Cuando repetimos estas iteraciones, los datos iniciales producen valoraciones progresivas del resultado.

4.8. Ejercicios y Tareas

1. Realizar una revisión de la historia de los métodos numéricos, elaborar un documento de hasta dos cuartillas.
2. Realiza las siguientes conversiones de base 10 a base 2:
 - a) 246
 - b) 345.68
 - c) 4586632.2846
 - d) 984365.27463
 - e) 79905523
3. Elabora el código en R para realizar la conversión de base 10 a base 2.

5. Solución de Sistemas de Ecuaciones Lineales

Un sistema de ecuaciones lineales puede escribirse como:

$$Ax = b, \quad A = (a_{ij}) \in \mathbb{R}^{n \times n}, \quad x, b \in \mathbb{R}^n.$$

Nuestro objetivo es encontrar x . La eliminación gaussiana consiste en aplicar operaciones de filas equivalentes que transforman A en una matriz triangular superior U , de modo que el sistema resultante pueda resolverse por sustitución regresiva.

5.1. Eliminación gaussiana simple

En el paso k de la eliminación:

$$a_{ij}^{(k+1)} = a_{ij}^{(k)} - m_{ik}a_{kj}^{(k)}, \quad b_i^{(k+1)} = b_i^{(k)} - m_{ik}b_k^{(k)},$$

para $i = k + 1, \dots, n$ y $j = k, \dots, n$, donde

$$m_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}.$$

Es decir:

- m_{ik} mide cuántas veces la fila k debe restarse a la fila i para anular a_{ik} .
- Cada m_{ik} se almacena en la matriz L .

5.2. Construcción de L y U

$$L = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ m_{21} & 1 & \cdots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ m_{n1} & \cdots & m_{n,n-1} & 1 \end{bmatrix}, \quad U = \text{matriz triangular superior obtenida tras la eliminación.}$$

Por tanto:

$$A = LU.$$

5.3. Pivoteo y escalamiento

- **Pivoteo parcial:** intercambiar la fila k con la fila p tal que $|a_{pk}^{(k)}| = \max_{i \geq k} |a_{ik}^{(k)}|$.
- **Escalamiento:** definir factores $s_i = \max_j |a_{ij}|$ y escoger como pivote el mayor cociente

$$\frac{|a_{ik}^{(k)}|}{s_i}.$$

Esto controla la propagación de errores de redondeo.

5.4. Gauss–Jordan e inversa

Si extendemos la eliminación a toda la matriz (no solo a triangular superior), obtenemos:

$$A \longrightarrow I, \quad (A|I) \longrightarrow (I|A^{-1}).$$

Algebraicamente:

$$A^{-1} = M^{(n-1)^{-1}} \dots M^{(2)^{-1}} M^{(1)^{-1}},$$

donde cada $M^{(k)}$ es la matriz elemental usada en la etapa k de la eliminación.

5.5. Eliminación Gaussiana Simple

Idea general. Transformar el sistema $Ax = b$ en uno triangular superior $Ux = c$ mediante operaciones elementales de filas, y resolver luego por sustitución regresiva.

Ejemplo. Resolver el sistema

$$\begin{cases} 2x + y - z = 8, \\ -3x - y + 2z = -11, \\ -2x + y + 2z = -3. \end{cases}$$

Forma matricial:

$$\begin{bmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 8 \\ -11 \\ -3 \end{bmatrix}.$$

Paso 1. Pivote en $a_{11} = 2$. Eliminamos en la primera columna:

$$m_{21} = \frac{-3}{2}, \quad m_{31} = \frac{-2}{2} = -1.$$

Se obtiene

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & 0.5 & 0.5 & 1 \\ 0 & 2 & 1 & 5 \end{array} \right].$$

Paso 2. Pivote en $a_{22} = 0.5$. Eliminamos debajo:

$$m_{32} = \frac{2}{0.5} = 4.$$

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & 0.5 & 0.5 & 1 \\ 0 & 0 & -1 & 1 \end{array} \right].$$

Paso 3. Sustitución regresiva:

$$z = -1, \quad 0.5y + 0.5z = 1 \Rightarrow y = 3, \quad 2x + y - z = 8 \Rightarrow x = 2.$$

$$\boxed{x = 2, y = 3, z = -1}.$$

5.6. Eliminación Gaussiana con Pivoteo Parcial

Idea. Si en algún paso el pivote es 0 o muy pequeño, se intercambia la fila pivote con otra fila inferior que tenga el mayor valor absoluto en la misma columna. Esto aumenta la estabilidad.

Ejemplo.

$$\left[\begin{array}{ccc|c} 0 & 2 & 1 & 4 \\ 1 & -1 & 2 & 6 \\ 2 & 1 & -1 & 1 \end{array} \right].$$

Problema: $a_{11} = 0$. *Solución:* intercambiamos fila 1 con fila 2.

Nuevo sistema:

$$\left[\begin{array}{ccc|c} 1 & -1 & 2 & 6 \\ 0 & 2 & 1 & 4 \\ 2 & 1 & -1 & 1 \end{array} \right].$$

Ahora se procede con eliminación gaussiana simple. El pivoteo evita la división por cero.

5.7. Eliminación con Pivoteo y Escalamiento

Idea. Si los coeficientes varían mucho en magnitud, usar sólo pivoteo puede ser insuficiente. *Escalamiento:* dividir cada fila por su elemento de mayor valor absoluto antes de seleccionar pivote. Así, se compara $|a_{ik}|/s_i$, donde $s_i = \max_j |a_{ij}|$ es el factor de escala de la fila i .

Comentario. Esto evita que números muy grandes o muy pequeños enmascaren el pivote real, aumentando la precisión numérica en cómputo.

5.8. Gauss–Jordan y cálculo de inversas

Idea. Extender la eliminación hasta reducir A a la matriz identidad. Si se parte de la matriz aumentada $(A|I)$, el resultado final es $(I|A^{-1})$.

Ejemplo. Calcular A^{-1} con

$$A = \begin{bmatrix} 2 & 1 \\ 5 & 3 \end{bmatrix}.$$

Matriz aumentada:

$$\left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 5 & 3 & 0 & 1 \end{array} \right].$$

Paso 1. Pivote 2. Normalizar fila 1:

$$\left[\begin{array}{cc|cc} 1 & 0.5 & 0.5 & 0 \\ 5 & 3 & 0 & 1 \end{array} \right].$$

Paso 2. Eliminar columna 1 de fila 2:

$$R_2 \leftarrow R_2 - 5R_1.$$
$$\left[\begin{array}{cc|cc} 1 & 0.5 & 0.5 & 0 \\ 0 & 0.5 & -2.5 & 1 \end{array} \right].$$

Paso 3. Normalizar fila 2:

$$\left[\begin{array}{cc|cc} 1 & 0.5 & 0.5 & 0 \\ 0 & 1 & -5 & 2 \end{array} \right].$$

Paso 4. Eliminar columna 2 de fila 1:

$$R_1 \leftarrow R_1 - 0.5R_2.$$
$$\left[\begin{array}{cc|cc} 1 & 0 & 3 & -1 \\ 0 & 1 & -5 & 2 \end{array} \right].$$

Resultado.

$$A^{-1} = \begin{bmatrix} 3 & -1 \\ -5 & 2 \end{bmatrix}.$$

5.9. Sustitución hacia adelante y hacia atrás

En los métodos directos como LU o Cholesky, al factorizar la matriz A en dos bloques (p.ej. $A = LU$ o $A = LL^T$), la resolución de $Ax = b$ se divide en dos etapas fundamentales:

1. Sustitución hacia adelante (forward substitution). Se aplica cuando se resuelve un sistema triangular inferior:

$$Ly = b, \quad L = (\ell_{ij}) \text{ triangular inferior con } \ell_{ii} \neq 0.$$

El proceso es:

$$y_1 = \frac{b_1}{\ell_{11}}, \quad y_i = \frac{1}{\ell_{ii}} \left(b_i - \sum_{j=1}^{i-1} \ell_{ij} y_j \right), \quad i = 2, \dots, n.$$

2. Sustitución hacia atrás (backward substitution). Se aplica cuando se resuelve un sistema triangular superior:

$$Ux = y, \quad U = (u_{ij}) \text{ triangular superior con } u_{ii} \neq 0.$$

El proceso es:

$$x_n = \frac{y_n}{u_{nn}}, \quad x_i = \frac{1}{u_{ii}} \left(y_i - \sum_{j=i+1}^n u_{ij} x_j \right), \quad i = n-1, \dots, 1.$$

Ejemplo simbólico (sistema triangular 3×3). Sea

$$U = \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}.$$

La sustitución hacia atrás da:

$$x_3 = \frac{y_3}{u_{33}}, \quad x_2 = \frac{y_2 - u_{23}x_3}{u_{22}}, \quad x_1 = \frac{y_1 - u_{12}x_2 - u_{13}x_3}{u_{11}}.$$

Importancia.

- Aunque son rutinas muy sencillas, se usan constantemente en implementación de métodos directos.
- En librerías numéricas (como en **R**, **C**, **Fortran** o **Python/NumPy**) suelen estar optimizadas y se llaman explícitamente cada vez que se resuelve un sistema triangular.
- En cursos posteriores (p.ej. optimización numérica o métodos iterativos avanzados), estas rutinas aparecen como bloques básicos para implementar algoritmos más sofisticados, como Gradiente Conjugado.

5.10. Descomposición de A

Escribimos:

$$A = D + L + U,$$

donde:

$D = \text{diag}(a_{11}, \dots, a_{nn})$, L = parte estrictamente triangular inferior, U = parte estrictamente triangular superior

5.11. Esquema general

De $Ax = b$ obtenemos:

$$x = -D^{-1}(L + U)x + D^{-1}b,$$

lo que sugiere la iteración:

$$x^{(k+1)} = Tx^{(k)} + c, \quad T = -D^{-1}(L + U), \quad c = D^{-1}b.$$

5.12. Método de Jacobi

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j^{(k)} \right), \quad i = 1, \dots, n.$$

En forma matricial:

$$T_J = -D^{-1}(L + U), \quad c_J = D^{-1}b.$$

5.13. Método de Gauss–Seidel

En este caso se aprovechan los nuevos valores en cuanto están disponibles:

$$x^{(k+1)} = (D + L)^{-1}(-Ux^{(k)} + b).$$

Por componentes:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right).$$

5.14. Condiciones de convergencia

Ambos métodos convergen si el radio espectral de la matriz de iteración es menor que uno:

$$\rho(T) < 1.$$

Casos suficientes:

- A diagonalmente dominante $\Rightarrow$ Jacobi y Gauss–Seidel convergen.
- A simétrica definida positiva $\Rightarrow$ Gauss–Seidel converge.

5.15. Método SOR (Successive Over-Relaxation)

Es una generalización de Gauss–Seidel con parámetro $\omega > 0$:

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right).$$

- $\omega = 1$ reproduce Gauss–Seidel.
- $1 < \omega < 2$: *sobrerrelajación*, posible aceleración.
- $0 < \omega < 1$: *sub-relajación*, usada en algunos problemas mal condicionados.

5.16. Eliminación Gaussiana Simple

Planteamiento. Sea $A = (a_{ij}) \in \mathbb{R}^{n \times n}$ y $b = (b_i) \in \mathbb{R}^n$. El sistema $Ax = b$ se expresa como:

$$\sum_{j=1}^n a_{ij}x_j = b_i, \quad i = 1, \dots, n.$$

Etapas de eliminación. En el paso k se anulan las entradas de la columna k por debajo del pivote $a_{kk}^{(k)}$. Para cada $i = k + 1, \dots, n$:

$$m_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}, \quad a_{ij}^{(k+1)} = a_{ij}^{(k)} - m_{ik}a_{kj}^{(k)}, \quad b_i^{(k+1)} = b_i^{(k)} - m_{ik}b_k^{(k)}.$$

Ejemplo simbólico 3×3 .

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}.$$

Paso 1 ($k = 1$):

$$m_{21} = \frac{a_{21}}{a_{11}}, \quad m_{31} = \frac{a_{31}}{a_{11}}.$$

$$a_{22}^{(2)} = a_{22} - m_{21}a_{12}, \quad a_{23}^{(2)} = a_{23} - m_{21}a_{13}, \quad b_2^{(2)} = b_2 - m_{21}b_1,$$

$$a_{32}^{(2)} = a_{32} - m_{31}a_{12}, \quad a_{33}^{(2)} = a_{33} - m_{31}a_{13}, \quad b_3^{(2)} = b_3 - m_{31}b_1.$$

Paso 2 ($k = 2$):

$$m_{32} = \frac{a_{32}^{(2)}}{a_{22}^{(2)}}, \quad a_{33}^{(3)} = a_{33}^{(2)} - m_{32}a_{23}^{(2)}, \quad b_3^{(3)} = b_3^{(2)} - m_{32}b_2^{(2)}.$$

Resultado: sistema triangular superior $Ux = c$.

5.17. Eliminación con pivoteo parcial

Mismo proceso, pero en cada paso k se intercambia la fila k con la fila p tal que

$$|a_{pk}^{(k)}| = \max_{i \geq k} |a_{ik}^{(k)}|.$$

Esto asegura que $a_{kk}^{(k)}$ sea lo suficientemente grande.

5.18. Eliminación con pivoteo y escalamiento

Antes de seleccionar pivote, cada fila i se escala por

$$s_i = \max_j |a_{ij}|,$$

y se elige como pivote aquel que maximiza

$$\frac{|a_{ik}^{(k)}|}{s_i}.$$

5.19. Gauss–Jordan e inversas

En vez de detener la eliminación en forma triangular superior, se continúa hasta obtener la matriz identidad:

$$(A|I) \longrightarrow (I|A^{-1}).$$

Así, cada paso se formaliza como multiplicación por matrices elementales:

$$A^{-1} = M_1^{-1} M_2^{-1} \cdots M_r^{-1}.$$

5.20. Descomposición de A

Sea $A = D + L + U$, con:

$D = \text{diag}(a_{11}, \dots, a_{nn})$, L = parte estrictamente triangular inferior, U = parte estrictamente triangular superior

5.21. Forma general de iteración

Reescribiendo $Ax = b$:

$$x = Tx + c, \quad T = -D^{-1}(L + U), \quad c = D^{-1}b.$$

De aquí nace el esquema iterativo:

$$x^{(k+1)} = Tx^{(k)} + c.$$

5.22. Método de Jacobi

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j^{(k)} \right).$$

Se calcula cada $x_i^{(k+1)}$ sólo con valores de la iteración anterior.

Ejemplo simbólico 3×3 .

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 = b_1, \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 = b_2, \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 = b_3. \end{cases}$$

Iteración:

$$x_1^{(k+1)} = \frac{1}{a_{11}}(b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)}), \quad x_2^{(k+1)} = \frac{1}{a_{22}}(b_2 - a_{21}x_1^{(k)} - a_{23}x_3^{(k)}), \dots$$

5.23. Método de Gauss–Seidel

Aprovecha los valores nuevos tan pronto como se calculan:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right).$$

5.24. Método SOR

Generaliza Gauss–Seidel:

$$x_i^{(k+1)} = (1 - \omega) x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right).$$

5.25. Convergencia

- Convergencia $\iff \rho(T) < 1$, donde ρ es el radio espectral.
- Suficiente: A diagonalmente dominante estricta $\Rightarrow$ Jacobi y GS convergen.
- Suficiente: A simétrica definida positiva $\Rightarrow$ GS siempre converge.

5.26. Resumen con matrices de iteración

$$\begin{aligned} T_J &= -D^{-1}(L + U), & c_J &= D^{-1}b, \\ T_{GS} &= -(D + L)^{-1}U, & c_{GS} &= (D + L)^{-1}b, \\ T_{SOR} &= (D + \omega L)^{-1}((1 - \omega)D - \omega U), & c_{SOR} &= \omega(D + \omega L)^{-1}b. \end{aligned}$$

5.27. Factorización LU

La **factorización LU** consiste en descomponer una matriz cuadrada

$$A \in \mathbb{R}^{n \times n}$$

en el producto de dos matrices:

$$A = LU,$$

donde:

- L es una matriz **triangular inferior** con unos en la diagonal.
- U es una matriz **triangular superior**.

En algunos casos es necesario introducir una matriz de permutación P para realizar el proceso de eliminación gaussiana de manera estable:

$$PA = LU.$$

5.27.1. Método general (sin permutaciones)

El procedimiento se basa en la eliminación gaussiana. Dada

$$A = (a_{ij}) \in \mathbb{R}^{n \times n},$$

se definen los **multiplicadores**:

$$m_{ij} = \frac{a_{ij}^{(k)}}{a_{kk}^{(k)}} \quad \text{para } i > k,$$

donde $a_{ij}^{(k)}$ denota la entrada de la matriz en la etapa k de la eliminación.

Los pasos son:

1. Inicialmente $L = I_n$ y $U = A$.
2. Para cada paso $k = 1, 2, \dots, n - 1$:
 - a) Calcular los multiplicadores

$$l_{ik} = \frac{u_{ik}}{u_{kk}}, \quad i = k + 1, \dots, n.$$

- b) Restar l_{ik} veces la fila k a la fila i de U .
- c) Almacenar cada l_{ik} en la posición correspondiente de L .

De este modo se obtiene:

$$A = LU, \quad L = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ l_{21} & 1 & 0 & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ l_{n1} & \cdots & l_{n,n-1} & 1 & \end{bmatrix}, \quad U = \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & u_{nn} \end{bmatrix}.$$

5.27.2. Método con permutaciones

Si en algún paso se cumple $u_{kk} = 0$, el algoritmo falla. Para evitarlo se aplica **pivoteo parcial**: se intercambia la fila k con otra fila $p > k$ tal que

$$|u_{pk}| = \max_{i \geq k} |u_{ik}|.$$

Esto se representa mediante una matriz de permutación P , resultando:

$$PA = LU.$$

5.27.3. Resumen

- La factorización LU es el corazón de la eliminación gaussiana.
- Permite resolver sistemas lineales $Ax = b$ mediante:

$$Ax = b \quad \Rightarrow \quad LUx = b.$$

- Se resuelve en dos etapas:

$$Ly = b, \quad Ux = y,$$

usando sustitución progresiva y regresiva.

5.27.4. Ejemplo 1: $A = LU$ (sin permutaciones)

Sea

$$A = \begin{bmatrix} 2 & 3 & 1 \\ 4 & 7 & 7 \\ -2 & 4 & 5 \end{bmatrix}.$$

Aplicamos eliminación gaussiana guardando los multiplicadores en L (con 1's en la diagonal).

Paso $k = 1$. Pivote $u_{11} = 2$. Multiplicadores:

$$\ell_{21} = \frac{4}{2} = 2, \quad \ell_{31} = \frac{-2}{2} = -1.$$

Actualizamos filas de U :

$$R_2 \leftarrow R_2 - 2R_1, \quad R_3 \leftarrow R_3 - (-1)R_1 = R_3 + R_1.$$

Esto produce

$$U^{(1)} = \begin{bmatrix} 2 & 3 & 1 \\ 0 & 1 & 5 \\ 0 & 7 & 6 \end{bmatrix}, \quad L^{(1)} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}.$$

Paso $k = 2$. Pivote $u_{22} = 1$. Multiplicador:

$$\ell_{32} = \frac{7}{1} = 7.$$

Actualizamos fila 3 de U :

$$R_3 \leftarrow R_3 - 7R_2 \implies U = \begin{bmatrix} 2 & 3 & 1 \\ 0 & 1 & 5 \\ 0 & 0 & -29 \end{bmatrix}.$$

Insertamos ℓ_{32} en L :

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 7 & 1 \end{bmatrix}.$$

Resultado. Se tiene

$$A = LU, \quad L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 7 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 2 & 3 & 1 \\ 0 & 1 & 5 \\ 0 & 0 & -29 \end{bmatrix}.$$

(Verificación rápida: $LU = A$ por multiplicación directa).

Uso para resolver $Ax = b$: Resolver $Ly = b$ (sustitución progresiva) y luego $Ux = y$ (sustitución regresiva).

5.27.5. Ejemplo 2: $PA = LU$ (con permutaciones)

Sea

$$A = \begin{bmatrix} 0 & 2 & 1 \\ 1 & -2 & -3 \\ -1 & 1 & 2 \end{bmatrix}.$$

El pivote inicial $a_{11} = 0$ es inválido; aplicamos **pivoteo parcial** intercambiando $R_1 \leftrightarrow R_2$. La matriz de permutación que permuta las primeras dos filas es

$$P = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad PA = \begin{bmatrix} 1 & -2 & -3 \\ 0 & 2 & 1 \\ -1 & 1 & 2 \end{bmatrix}.$$

Paso $k = 1$ sobre PA . Pivote $u_{11} = 1$. Multiplicador:

$$\ell_{31} = \frac{-1}{1} = -1.$$

Actualizamos fila 3:

$$R_3 \leftarrow R_3 - (-1)R_1 = R_3 + R_1 \implies U^{(1)} = \begin{bmatrix} 1 & -2 & -3 \\ 0 & 2 & 1 \\ 0 & -1 & -1 \end{bmatrix}, \quad L^{(1)} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}.$$

Paso $k = 2$. Pivote $u_{22} = 2$. Multiplicador:

$$\ell_{32} = \frac{-1}{2} = -\frac{1}{2}.$$

Actualizamos fila 3:

$$R_3 \leftarrow R_3 - (-\frac{1}{2})R_2 = R_3 + \frac{1}{2}R_2 \implies U = \begin{bmatrix} 1 & -2 & -3 \\ 0 & 2 & 1 \\ 0 & 0 & -\frac{1}{2} \end{bmatrix}.$$

Insertamos ℓ_{32} en L :

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -1 & -\frac{1}{2} & 1 \end{bmatrix}.$$

Resultado. Se obtiene

$$\boxed{PA = LU, \quad P = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -1 & -\frac{1}{2} & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 1 & -2 & -3 \\ 0 & 2 & 1 \\ 0 & 0 & -\frac{1}{2} \end{bmatrix}.$$

(Comprobación: $LU = PA$ y, por tanto, $A = P^T LU$ ya que $P^{-1} = P^T$.)

5.27.6. Ejemplo: Resolver $Ax = b$ vía LU

Consideremos

$$A = \begin{bmatrix} 2 & 3 & 1 \\ 4 & 7 & 7 \\ -2 & 4 & 5 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.$$

Ya hemos obtenido la factorización

$$A = LU, \quad L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 7 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 2 & 3 & 1 \\ 0 & 1 & 5 \\ 0 & 0 & -29 \end{bmatrix}.$$

El sistema $Ax = b$ se resuelve en dos etapas:

1. Resolver $Ly = b$.

$$\begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 7 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.$$

De aquí:

$$y_1 = 1, \quad 2y_1 + y_2 = 2 \Rightarrow y_2 = 0, \quad -1 \cdot y_1 + 7y_2 + y_3 = 3 \Rightarrow y_3 = 4.$$

Por lo tanto,

$$y = \begin{bmatrix} 1 \\ 0 \\ 4 \end{bmatrix}.$$

2. Resolver $Ux = y$.

$$\begin{bmatrix} 2 & 3 & 1 \\ 0 & 1 & 5 \\ 0 & 0 & -29 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 4 \end{bmatrix}.$$

De abajo hacia arriba:

$$-29x_3 = 4 \Rightarrow x_3 = -\frac{4}{29},$$

$$x_2 + 5x_3 = 0 \Rightarrow x_2 = -5x_3 = \frac{20}{29},$$

$$2x_1 + 3x_2 + x_3 = 1 \Rightarrow 2x_1 + 3 \cdot \frac{20}{29} - \frac{4}{29} = 1.$$

$$2x_1 + \frac{60-4}{29} = 1 \Rightarrow 2x_1 + \frac{56}{29} = 1.$$

$$2x_1 = \frac{29}{29} - \frac{56}{29} = -\frac{27}{29} \Rightarrow x_1 = -\frac{27}{58}.$$

Solución final:

$$x = \begin{bmatrix} -\frac{27}{58} \\ \frac{20}{29} \\ -\frac{4}{29} \end{bmatrix}.$$

5.28. Definiciones fundamentales de matrices

Definición 7 (Matriz transpuesta). Sea $A = (a_{ij}) \in \mathbb{R}^{m \times n}$. La **matriz transpuesta** de A , denotada $A^\top$, es la matriz $n \times m$ definida por

$$(A^\top)_{ij} = a_{ji}.$$

Definición 8 (Matriz simétrica). Una matriz $A \in \mathbb{R}^{n \times n}$ es **simétrica** si

$$A^\top = A.$$

Definición 9 (Matriz definida positiva). Una matriz simétrica $A \in \mathbb{R}^{n \times n}$ es **definida positiva** si

$$x^\top A x > 0 \quad \text{para todo } x \in \mathbb{R}^n, x \neq 0.$$

Definición 10 (Matriz semidefinida positiva). Una matriz simétrica $A \in \mathbb{R}^{n \times n}$ es **semidefinida positiva** si

$$x^\top A x \geq 0 \quad \text{para todo } x \in \mathbb{R}^n.$$

5.29. Factorización de Doolittle

Es una variante de la factorización LU en la cual la matriz L tiene 1's en la diagonal principal:

$$A = LU, \quad L \text{ con unos en la diagonal, } U \text{ triangular superior.}$$

5.30. Factorización de Cholesky

Si A es simétrica y definida positiva, entonces admite una factorización especial:

$$A = LL^\top,$$

donde L es triangular inferior con entradas reales y diagonales positivas. Esta factorización es más eficiente y estable que la LU en estos casos.

Recordatorio del método

Sea $A \in \mathbb{R}^{n \times n}$ simétrica definida positiva. La factorización de Cholesky busca

$$A = LL^\top,$$

con L triangular inferior y diagonal positiva. Las fórmulas recursivas son, para $i = 1, \dots, n$:

$$\ell_{ii} = \sqrt{a_{ii} - \sum_{k=1}^{i-1} \ell_{ik}^2}, \quad \ell_{ji} = \frac{1}{\ell_{ii}} \left(a_{ji} - \sum_{k=1}^{i-1} \ell_{jk} \ell_{ik} \right) \quad (j = i+1, \dots, n).$$

Ejemplo 1 (con entradas enteras)

Considérese

$$A = \begin{bmatrix} 4 & 2 & 2 \\ 2 & 2 & 1 \\ 2 & 1 & 2 \end{bmatrix}.$$

Es simétrica y definida positiva (sus menores principales son positivos). Apliquemos las fórmulas:

Columna $i = 1$.

$$\ell_{11} = \sqrt{4} = 2, \quad \ell_{21} = \frac{2}{2} = 1, \quad \ell_{31} = \frac{2}{2} = 1.$$

Columna $i = 2$.

$$\ell_{22} = \sqrt{2 - \ell_{21}^2} = \sqrt{2 - 1} = 1, \quad \ell_{32} = \frac{1 - \ell_{31}\ell_{21}}{\ell_{22}} = \frac{1 - 1 \cdot 1}{1} = 0.$$

Columna $i = 3$.

$$\ell_{33} = \sqrt{2 - \ell_{31}^2 - \ell_{32}^2} = \sqrt{2 - 1 - 0} = 1.$$

Resultado.

$$L = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}, \quad \boxed{A = LL^\top}.$$

(Verificación rápida: $LL^\top = \begin{bmatrix} 4 & 2 & 2 \\ 2 & 2 & 1 \\ 2 & 1 & 2 \end{bmatrix}$.)

Ejemplo 2 (con radical exacto)

Considérese

$$A = \begin{bmatrix} 9 & 3 & 6 \\ 3 & 5 & 3 \\ 6 & 3 & 14 \end{bmatrix}.$$

También es simétrica definida positiva. Procedamos:

Columna $i = 1$.

$$\ell_{11} = \sqrt{9} = 3, \quad \ell_{21} = \frac{3}{3} = 1, \quad \ell_{31} = \frac{6}{3} = 2.$$

Columna $i = 2$.

$$\ell_{22} = \sqrt{5 - \ell_{21}^2} = \sqrt{5 - 1} = 2, \quad \ell_{32} = \frac{3 - \ell_{31}\ell_{21}}{\ell_{22}} = \frac{3 - 2 \cdot 1}{2} = \frac{1}{2}.$$

Columna $i = 3$.

$$\ell_{33} = \sqrt{14 - \ell_{31}^2 - \ell_{32}^2} = \sqrt{14 - 4 - \frac{1}{4}} = \sqrt{\frac{39}{4}} = \frac{\sqrt{39}}{2}.$$

Resultado.

$$L = \begin{bmatrix} 3 & 0 & 0 \\ 1 & 2 & 0 \\ 2 & \frac{1}{2} & \frac{\sqrt{39}}{2} \end{bmatrix}, \quad \boxed{A = LL^\top}.$$

(Verificación: $LL^\top$ reproduce A ; la última diagonal verifica $2^2 + (\frac{1}{2})^2 + (\frac{\sqrt{39}}{2})^2 = 14$.)

5.31. Doolittle (LU)

- **Forma:** $A = LU$, con L triangular inferior con 1 en la diagonal y U triangular superior.
- **Requisitos:** Válida para matrices cuadradas no singulares (en la práctica, puede requerir permutaciones: $PA = LU$).
- **Pivoteo:** Suele emplear *pivoteo parcial* para estabilidad numérica.
- **Costo:** $\approx \frac{2}{3}n^3$ FLOPs en factorización; resolución de $Ax = b$ vía LU cuesta $\approx 2n^2$ FLOPs (sustitución hacia adelante y hacia atrás).
- **Cuándo usarla:** Sistemas generales (no necesariamente simétricos ni definidos positivos), múltiples RHS b con la misma A .

5.32. Cholesky ($LL^\top$)

- **Forma:** $A = LL^\top$, con L triangular inferior y diagonal positiva.
- **Requisitos:** A debe ser **simétrica definida positiva (SDP)**.
- **Pivoteo:** No requiere pivoteo si A es SDP (bien condicionada).
- **Costo:** $\approx \frac{1}{3}n^3$ FLOPs (mitad de LU); resolver $Ax = b$ cuesta $\approx 2n^2$ FLOPs (primero $Ly = b$, luego $L^\top x = y$).
- **Ventajas:** Más rápida y estable (sin pivoteo) para matrices SDP; mejor aprovechamiento de memoria (simetría).
- **Cuándo usarla:** Sistemas simétricos definidos positivos (p.ej. matrices de Gram, problemas de mínimos cuadrados regulares, discretizaciones elípticas).

Resumen práctico. Si A es SDP, use Cholesky. En caso general, use LU con pivoteo.

Esquema general

Si $A = LL^\top$ con L triangular inferior y diagonal positiva, resolver $Ax = b$ equivale a:

$$Ly = b \quad (\text{sustitución progresiva}), \quad L^\top x = y \quad (\text{sustitución regresiva}).$$

Ejemplo A (matriz del Ejemplo 1 de Cholesky)

Sea

$$A = \begin{bmatrix} 4 & 2 & 2 \\ 2 & 2 & 1 \\ 2 & 1 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.$$

Su factorización de Cholesky es

$$L = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \implies A = LL^\top.$$

Paso 1: $Ly = b$.

$$\begin{bmatrix} 2 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \Rightarrow \begin{cases} 2y_1 = 1 \Rightarrow y_1 = \frac{1}{2}, \\ y_1 + y_2 = 2 \Rightarrow y_2 = \frac{3}{2}, \\ y_1 + y_3 = 3 \Rightarrow y_3 = \frac{5}{2}. \end{cases}$$

Por tanto, $y = \left[\frac{1}{2}, \frac{3}{2}, \frac{5}{2}\right]^\top$.

Paso 2: $L^\top x = y$.

$$L^\top = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} \frac{1}{2} \\ \frac{3}{2} \\ \frac{5}{2} \end{bmatrix} \Rightarrow \begin{cases} x_3 = \frac{5}{2}, \\ x_2 = \frac{3}{2}, \\ 2x_1 + x_2 + x_3 = \frac{1}{2} \Rightarrow 2x_1 = \frac{1}{2} - \frac{3}{2} - \frac{5}{2} = -\frac{7}{2} \Rightarrow x_1 = -\frac{7}{4}. \end{cases}$$

$$x = \begin{bmatrix} -\frac{7}{4} \\ \frac{3}{2} \\ \frac{5}{2} \end{bmatrix} \quad (\text{verificación: } Ax = b).$$

Ejemplo B (matriz del Ejemplo 2 de Cholesky)

Sea

$$A = \begin{bmatrix} 9 & 3 & 6 \\ 3 & 5 & 3 \\ 6 & 3 & 14 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.$$

Una factorización de Cholesky es

$$L = \begin{bmatrix} 3 & 0 & 0 \\ 1 & 2 & 0 \\ 2 & \frac{1}{2} & \frac{\sqrt{39}}{2} \end{bmatrix} \implies A = LL^\top.$$

Paso 1: $Ly = b$.

$$\begin{bmatrix} 3 & 0 & 0 \\ 1 & 2 & 0 \\ 2 & \frac{1}{2} & \frac{\sqrt{39}}{2} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \Rightarrow \begin{cases} 3y_1 = 1 \Rightarrow y_1 = \frac{1}{3}, \\ y_1 + 2y_2 = 2 \Rightarrow y_2 = \frac{2 - \frac{1}{3}}{2} = \frac{5}{6}, \\ 2y_1 + \frac{1}{2}y_2 + \frac{\sqrt{39}}{2}y_3 = 3 \Rightarrow \frac{\sqrt{39}}{2}y_3 = 3 - \frac{2}{3} - \frac{5}{12} = \frac{23}{12} \\ \Rightarrow y_3 = \frac{23}{12} \cdot \frac{2}{\sqrt{39}} = \frac{23}{6\sqrt{39}}. \end{cases}$$

Por tanto, $y = \left[\frac{1}{3}, \frac{5}{6}, \frac{23}{6\sqrt{39}}\right]^\top$.

Paso 2: $L^\top x = y$.

$$L^\top = \begin{bmatrix} 3 & 1 & 2 \\ 0 & 2 & \frac{1}{2} \\ 0 & 0 & \frac{\sqrt{39}}{2} \end{bmatrix}, \quad \begin{bmatrix} 3 & 1 & 2 \\ 0 & 2 & \frac{1}{2} \\ 0 & 0 & \frac{\sqrt{39}}{2} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} \frac{1}{3} \\ \frac{5}{6} \\ \frac{23}{6\sqrt{39}} \end{bmatrix} \Rightarrow \begin{cases} \frac{\sqrt{39}}{2} x_3 = \frac{23}{6\sqrt{39}} \Rightarrow x_3 = \frac{23}{117}, \\ 2x_2 + \frac{1}{2}x_3 = \frac{5}{6} \Rightarrow 2x_2 = \frac{5}{6} - \frac{1}{2} \cdot \frac{23}{117} = \frac{86}{117} \Rightarrow x_2 = \frac{43}{117}, \\ 3x_1 + x_2 + 2x_3 = \frac{1}{3} = \frac{39}{117} \Rightarrow 3x_1 = \frac{39-43-46}{117} = -\frac{50}{117} \Rightarrow x_1 = -\frac{50}{351} \end{cases}$$

$$\boxed{x = \begin{bmatrix} -\frac{50}{351} \\ \frac{43}{117} \\ \frac{23}{117} \end{bmatrix}} \quad (\text{verificación: } Ax = b).$$

5.33. Métodos iterativos para resolver sistemas lineales

Hasta ahora hemos visto métodos **directos** (eliminación Gaussiana, factorizaciones LU , Doolittle, Cholesky). En problemas de gran tamaño, estos métodos pueden ser muy costosos en tiempo y memoria. Por ello se usan **métodos iterativos**, que generan una sucesión de aproximaciones $\{x^{(k)}\}$ que converge a la solución exacta x de

$$Ax = b.$$

5.33.1. Descomposición de la matriz

Sea $A \in \mathbb{R}^{n \times n}$. Se descompone como

$$A = D - L - U,$$

donde:

- D : matriz diagonal de A ,
- $-L$: parte estrictamente triangular inferior,
- $-U$: parte estrictamente triangular superior.

Así,

$$A = D + L + U, \quad Ax = b \iff (D + L + U)x = b.$$

5.34. Método de Jacobi

A partir de la descomposición, se obtiene el esquema iterativo de Jacobi:

$$x^{(k+1)} = D^{-1}(b - (L + U)x^{(k)}).$$

De manera componente a componente:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j^{(k)} \right), \quad i = 1, \dots, n.$$

- Cada componente $x_i^{(k+1)}$ se calcula usando exclusivamente valores de la iteración previa $x^{(k)}$.
- Es fácil de implementar en paralelo.

5.35. Método de Gauss–Seidel

En Gauss–Seidel se aprovecha que, en cada paso, los nuevos valores calculados pueden usarse de inmediato:

$$x^{(k+1)} = (D + L)^{-1}(b - Ux^{(k)}).$$

En forma componente:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right).$$

- La diferencia clave con Jacobi: $x_j^{(k+1)}$ ya calculados se usan de inmediato.
- Suele converger más rápido que Jacobi.

5.36. Condiciones de convergencia

Un resultado clásico:

- Si A es **diagonal dominante estricta** (es decir, $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ para todo i), entonces Jacobi y Gauss–Seidel convergen.
- Si A es **simétrica definida positiva**, Gauss–Seidel siempre converge.
- En general, la convergencia depende de que el radio espectral de la matriz de iteración sea menor que 1:

$$\rho(T) < 1.$$

5.37. Esquema general

1. Elegir un vector inicial $x^{(0)}$ (p.ej. el vector nulo).
2. Repetir hasta convergencia:
 - a) Calcular $x^{(k+1)}$ según el método elegido (Jacobi o Gauss–Seidel).
 - b) Medir el error, p.ej. $\|x^{(k+1)} - x^{(k)}\|$.
 - c) Detenerse cuando el error sea menor que una tolerancia ε .

5.37.1. Ejemplos

En clase se mostrarán ejemplos numéricos con matrices pequeñas, donde se puedan seguir las iteraciones:

- Resolver $Ax = b$ con Jacobi.
- Resolver el mismo sistema con Gauss–Seidel.
- Comparar número de iteraciones y velocidad de convergencia.

5.37.2. Sistema de prueba (diagonalmente dominante)

Consideremos

$$A = \begin{bmatrix} 10 & -1 & 2 & 0 \\ -1 & 11 & -1 & 3 \\ 2 & -1 & 10 & -1 \\ 0 & 3 & -1 & 8 \end{bmatrix}, \quad b = \begin{bmatrix} 6 \\ 25 \\ -11 \\ 15 \end{bmatrix},$$

con vector inicial $x^{(0)} = \mathbf{0}$. Este sistema es *diagonalmente dominante*, por lo que Jacobi y Gauss–Seidel convergen.

La solución exacta es

$$x^* = \begin{bmatrix} 1 \\ 2 \\ -1 \\ 1 \end{bmatrix}.$$

5.37.3. Método de Jacobi

$$x^{(k+1)} = D^{-1}(b - (L + U)x^{(k)}), \quad x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right).$$

Iteraciones (redondeadas a 6 decimales).

$$x^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad x^{(1)} = \begin{bmatrix} 0.6 \\ 2.272727 \\ -1.100000 \\ 1.875000 \end{bmatrix}, \quad x^{(2)} = \begin{bmatrix} 1.047273 \\ 1.715909 \\ -0.805227 \\ 0.885227 \end{bmatrix}, \quad x^{(3)} = \begin{bmatrix} 0.932636 \\ 2.053306 \\ -1.049341 \\ 1.130881 \end{bmatrix}.$$

Errores (norma infinito) frente a x^* .

$$\|x^{(0)} - x^*\|_\infty = 2.000000, \quad \|x^{(1)} - x^*\|_\infty = 0.875000, \quad \|x^{(2)} - x^*\|_\infty = 0.284091, \quad \|x^{(3)} - x^*\|_\infty = 0.130881.$$

5.37.4. Método de Gauss–Seidel

$$x^{(k+1)} = (D + L)^{-1}(b - Ux^{(k)}), \quad x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right).$$

Iteraciones (redondeadas a 6 decimales).

$$x^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad x^{(1)} = \begin{bmatrix} 0.6 \\ 2.327273 \\ -0.987273 \\ 0.878864 \end{bmatrix}, \quad x^{(2)} = \begin{bmatrix} 1.030182 \\ 2.036938 \\ -1.014456 \\ 0.984341 \end{bmatrix}, \quad x^{(3)} = \begin{bmatrix} 1.006585 \\ 2.003555 \\ -1.002527 \\ 0.998351 \end{bmatrix}.$$

Errores (norma infinito) frente a x^* .

$$\|x^{(0)} - x^*\|_\infty = 2.000000, \quad \|x^{(1)} - x^*\|_\infty = 0.400000, \quad \|x^{(2)} - x^*\|_\infty = 0.036938, \quad \|x^{(3)} - x^*\|_\infty = 0.006585.$$

- Ambos métodos convergen (la matriz es diagonalmente dominante), pero **Gauss–Seidel** reduce el error notablemente más rápido al reutilizar los valores nuevos dentro de la misma iteración.
- Criterio de paro típico: detener cuando $\|x^{(k+1)} - x^{(k)}\| \leq \varepsilon$ o $\|Ax^{(k)} - b\| \leq \varepsilon$.
- Para paralelización masiva, **Jacobi** es más simple; para rapidez en CPU única, **Gauss–Seidel** suele ser preferible.

5.38. Método de Relajación Sucesiva (SOR)

El método de **relajación sucesiva** o *Successive Over-Relaxation (SOR)* es una variante del método de Gauss–Seidel que introduce un parámetro $\omega > 0$, llamado **parámetro de relajación**.

Esquema general

Dado $Ax = b$ y un vector inicial $x^{(0)}$, el método SOR genera la sucesión:

$$x_i^{(k+1)} = (1 - \omega) x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right), \quad i = 1, \dots, n.$$

- Si $\omega = 1$, se recupera exactamente el método de Gauss–Seidel.
- Si $0 < \omega < 1$, se llama *relajación sub-sucesiva*, útil para estabilizar ciertos casos.
- Si $1 < \omega < 2$, se llama *sobrerrelajación*, y puede acelerar notablemente la convergencia.
- La elección óptima de ω depende de la matriz A ; en la práctica, se prueba con valores como $\omega \approx 1.1$ a 1.5 .
- El método es útil en problemas grandes, como los provenientes de discretización de ecuaciones diferenciales.
- Jacobi: usa valores viejos.
- Gauss–Seidel: usa valores nuevos en cuanto están disponibles.
- SOR: Gauss–Seidel + parámetro ω que acelera la convergencia si se elige bien.

5.39. SOR en una sola iteración (comparación con Gauss–Seidel)

Consideremos el mismo sistema diagonalmente dominante:

$$A = \begin{bmatrix} 10 & -1 & 2 & 0 \\ -1 & 11 & -1 & 3 \\ 2 & -1 & 10 & -1 \\ 0 & 3 & -1 & 8 \end{bmatrix}, \quad b = \begin{bmatrix} 6 \\ 25 \\ -11 \\ 15 \end{bmatrix}, \quad x^{(0)} = \mathbf{0}.$$

Esquema SOR (con $\omega = 1.2$)

$$x_i^{(k+1)} = (1 - \omega) x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right).$$

Una iteración desde $x^{(0)} = 0$ (redondeado a 6 decimales).

$$x_{\text{SOR}}^{(1)} = \begin{bmatrix} 0.720000 \\ 2.805818 \\ -1.156102 \\ 0.813967 \end{bmatrix}.$$

Referencia: Gauss–Seidel, una iteración

$$x_{\text{GS}}^{(1)} = \begin{bmatrix} 0.600000 \\ 2.327273 \\ -0.987273 \\ 0.878864 \end{bmatrix}.$$

- Con $\omega > 1$ (sobrerrelajación), SOR puede *acelerar* la convergencia, pero la **primera** iteración puede mostrar un “sobre-disparo” respecto a la solución exacta.
- La elección de ω es clave: en la práctica se prueba $\omega \in [1.1, 1.5]$ y se observa el comportamiento; si ω es muy grande, puede empeorar o incluso hacer divergir el método.
- Si A es SPD, métodos como **Cholesky** (directo) o iterativos tipo **Gradiente Conjugado** suelen ser preferibles por estabilidad y rapidez.

5.40. Ejercicios

5.40.1. Ejercicios

Ejercicio 3. Resolver por eliminación Gaussiana Simple los siguientes sistemas de ecuaciones lineales

1.

$$\begin{aligned} x_1 - 2x_2 + 0.5x_3 &= -5 \\ -2x_1 + 5x_2 - 1.5x_3 &= 0 \\ -0.2x_1 + 1.75x_2 - x_3 &= 10 \end{aligned}$$

2.

$$\begin{aligned} 3x_1 - x_2 + 6x_4 &= 2.3 \\ 4x_1 + 2x_2 - x_3 - 5x_4 &= 6.9 \\ -5x_1 + x_2 - 3x_3 &= -36 \\ 10x_2 - 4x_3 + 7x_4 &= -36 \end{aligned}$$

3.

$$\begin{aligned} 0.003000x_1 + 59.14x_2 &= 59.17 \\ 5.291x_1 - 6.130x_2 &= 46.78 \end{aligned}$$

utilizar redondeo a 4 cifras significativas.

4.

$$\begin{aligned} 4x_1 + 2x_2 &= 2 \\ 2x_1 + 3x_2 + x_3 &= -1 \\ x_2 + \frac{5}{2}x_3 &= 3 \end{aligned}$$

5.

$$\begin{aligned} 3x_1 - 0.1x_2 - 0.2x_3 &= 7.85 \\ 0.1x_1 + 7x_2 - 0.3x_3 &= -19.3 \\ 0.3x_1 - 0.2x_2 + 10x_3 &= 71.4 \end{aligned}$$

6.

$$\begin{aligned} 8x_1 + 2x_2 - 2x_3 &= -2 \\ 10x_1 + 2x_2 + 4x_3 &= 4 \\ 12x_1 + 2x_2 + 2x_3 &= 6 \end{aligned}$$

7.

$$\begin{aligned} 5x_1 + 2x_2 + x_3 - x_4 &= 1 \\ 2x_1 - x_2 + 3x_3 + 2x_4 &= 12 \\ 4x_1 + x_2 - 2x_3 + 3x_4 &= 5 \\ -2x_1 + 2x_2 + x_3 + x_4 &= 2 \end{aligned}$$

8.

$$\begin{aligned} 2x_1 + 3x_2 + 2x_3 + 4x_4 &= 4 \\ 4x_1 + 10x_2 - 4x_3 &= -8 \\ -3x_1 - 2x_2 - 5x_3 - 2x_4 &= -4 \\ -2x_1 + 4x_2 + 4x_3 - 7x_4 &= -1 \end{aligned}$$

9.

$$\begin{aligned} 1.133x_1 + 5.281x_2 - 2.454x_3 &= 6.414 \\ 24.14x_1 - 1.21x_2 + 5.281x_3 &= 113.8 \\ -10.123x_1 + 6.387x_2 - x_3 &= 1 \end{aligned}$$

$$10. \quad A = \begin{pmatrix} 2 & 3 & 2 & 4 \\ 4 & 10 & -4 & 0 \\ -3 & -2 & -5 & -2 \\ -2 & 4 & 4 & -7 \end{pmatrix} \quad y \quad b = \begin{pmatrix} 9 \\ -15 \\ 6 \\ 2 \end{pmatrix}$$

5.40.2. Ejercicios

Ejercicio 4. Resolver por eliminación gaussiana con pivoteo parcial los siguientes sistemas de ecuaciones lineales

1.

$$\begin{aligned} 0.4x_1 - 1.5x_2 + 0.75x_3 &= -20 \\ -0.5x_1 - 15x_2 + 10x_3 &= -10 \\ -10x_1 - 9x_2 + 2.5x_3 &= 30 \end{aligned}$$

2.

$$\begin{aligned} 5x_1 - 8x_2 + x_3 &= -71 \\ -2x_1 + 6x_2 - 9x_3 &= 134 \\ 3x_1 - 5x_2 + 2x_3 &= -58 \end{aligned}$$

3.

$$\begin{aligned} 0.003000x_1 + 59.14x_2 &= 59.17 \\ 5.291x_1 - 6.130x_2 &= 46.78 \end{aligned}$$

utilizar redondeo a 4 cifras significativas.

4.

$$\begin{aligned} -x_2 + 4x_3 - x_4 &= -1 \\ -x_1 + 4x_2 - x_3 &= 2 \\ -x_1 - x_3 + 4x_4 &= 4 \\ 4x_1 - x_2 &= 10 \end{aligned}$$

5.

$$\begin{aligned} 0.00031000x_1 + 1.000000x_2 &= 3.000000 \\ 1.00045534x_1 + 1.00034333x_2 &= 7.000 \end{aligned}$$

6. Resolver para $A = \begin{pmatrix} 14 & 14 & -9 & 3 & -5 \\ 14 & 52 & -15 & 2 & -32 \\ -9 & -15 & 36 & -5 & 16 \\ 3 & 2 & -5 & 47 & 49 \\ -5 & 32 & 16 & 49 & 79 \end{pmatrix}$, $b = \begin{pmatrix} -15 \\ -100 \\ 106 \\ 329 \\ 463 \end{pmatrix}$ y $X = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix}$

7. Resolver para $A = \begin{pmatrix} \frac{1}{4} & \frac{1}{5} & \frac{1}{6} \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{5} \\ \frac{3}{2} & 1 & 2 \end{pmatrix}$, $b = \begin{pmatrix} 9 \\ 8 \\ 8 \end{pmatrix}$ y $X = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$

8. Resolver para $A = \begin{pmatrix} 1 & \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \\ \frac{3}{1} & \frac{4}{1} & \frac{5}{1} & \frac{6}{1} \\ \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} \end{pmatrix}$, $b = \begin{pmatrix} \frac{1}{6} \\ \frac{1}{7} \\ \frac{1}{8} \\ \frac{1}{9} \end{pmatrix}$ y $X = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix}$

9. Resolver el sistema

$$\begin{aligned}2x_1 + x_2 - x_3 + x_4 - 3x_5 &= 7 \\x_1 + 2x_3 - x_4 + x_5 &= 2 \\-2x_2 - x_3 + x_4 - x_5 &= -5 \\3x_1 + x_2 - 4x_3 + 5x_5 &= 6 \\x_1 - x_2 - x_3 - x_4 + x_5 &= 3\end{aligned}$$

10. Resolver el sistema

$$\begin{aligned}3.333x_1 + 15920x_2 - 10.333x_3 &= 15913 \\2.222x_1 + 16.71x_2 + 9.612x_3 &= 28.544 \\1.5611x_1 + 5.1791x_2 + 1.6852x_3 &= 8.4254\end{aligned}$$

5.40.3. Ejercicios

Ejercicio 5. Resolver por el método de Gauss-Jordan los siguientes sistemas de ecuaciones lineales

1.

$$\begin{aligned}x_1 + 2x_2 + 3x_3 &= 1 \\-0.4x_1 + 2x_2 - x_3 &= 10 \\0.5x_1 - 3x_2 + x_3 &= 15\end{aligned}$$

2.

$$\begin{aligned}2x_1 - 0.9x_2 + 3x_3 &= -3.61 \\-0.5x_1 + 0.1x_2 - x_3 &= 2.035 \\x_1 - 6.35x_2 - 0.45x_3 &= 15.401\end{aligned}$$

3.

$$\begin{aligned}0.7x_1 + 2.7x_2 - 6x_3 + 0.7x_4 &= 1.6487 \\2x_1 - 0.8x_2 + 3x_3 - x_4 &= -2.342 \\-x_1 - 1.5x_2 + 1.4x_3 + 3x_4 &= -4.189 \\7x_2 - 1.56x_3 + x_4 &= 15.792\end{aligned}$$

4.

$$\begin{aligned}3x_1 - 0.1x_2 - 0.2x_3 &= 7.85 \\0.1x_1 + 7x_2 - 0.3x_3 &= -19.3 \\0.3x_1 - 0.2x_2 + 10x_3 &= 71.4\end{aligned}$$

5.

$$\begin{aligned}10x_1 + 2x_2 - x_3 &= 27 \\-3x_1 - 6x_2 + 2x_3 &= -61.5 \\x_1 + x_2 + 5x_3 &= -21.5\end{aligned}$$

$$6. \quad A = \begin{pmatrix} 1 & 3 & -2 & 1 \\ 1 & 3 & -1 & 2 \\ 0 & 1 & -1 & 4 \\ 2 & 6 & 1 & 2 \end{pmatrix} \quad y \quad b = \begin{pmatrix} 4 \\ 1 \\ 5 \\ 2 \end{pmatrix}$$

7.

$$\begin{aligned} 6x_1 - x_2 - x_3 + 4x_4 &= 17 \\ x_1 - 10x_2 + 2x_3 - x_4 &= -17 \\ 3x_1 - 2x_2 + 8x_3 - x_4 &= 19 \\ x_1 + x_2 + x_3 - 5x_4 &= -14 \end{aligned}$$

8.

$$\begin{aligned} x + 2y + 3z + 4w &= 1 \\ x - 4y + z + 11w &= 2 \\ -x + 8y + 7z + 6w &= -2 \\ 16x + 8y - 5z + 6w &= 11 \end{aligned}$$

9.

$$\begin{aligned} x_1 + x_2 &= 3 \\ x_1 + 2x_2 + x_3 &= -1 \\ x_2 + 3x_3 + x_4 &= 2 \\ x_3 + 4x_4 + x_5 &= 1 \\ x_4 + 5x_5 &= 3 \end{aligned}$$

10.

$$\begin{aligned} 15x_1 - 18x_2 + 15x_3 - 3x_4 &= 11 \\ -18x_1 + 24x_2 - 18x_3 + 4x_4 &= 10 \\ 15x_1 - 18x_2 + 18x_3 - 3x_4 &= 11 \\ -3x_1 + 4x_2 - 3x_3 + x_4 &= 13 \end{aligned}$$

5.40.4. Ejercicios

Ejercicio 6. Resolver por el método de Gauss-Seidel los siguientes sistemas de ecuaciones lineales

1.

$$\begin{aligned} 3x_1 - 0.2x_2 - 0.5x_3 &= 8 \\ 0.1x_1 + 7x_2 + 0.4x_3 &= -19.5 \\ 0.4x_1 - 0.1x_2 + 10x_3 &= 72.4 \end{aligned}$$

2.

$$\begin{aligned} -5x_1 + 1.4x_2 - 2.7x_3 &= 94.2 \\ 0.7x_1 - 2.5x_2 + 15x_3 &= -6 \\ 3.3x_1 - 11x_2 + 4.4x_3 &= -27.5 \end{aligned}$$

3.

$$\begin{aligned}3x_1 - 0.5x_2 + 0.6x_3 &= 5.24 \\0.3x_1 - 4x_2 - x_3 &= -0.387 \\-0.7x_1 + 2x_2 + 7x_3 &= 14.803\end{aligned}$$

4.

$$\begin{aligned}5x_1 - 0.2x_2 + x_3 &= 1.5 \\0.1x_1 + 3x_2 - 0.5x_3 &= -2.7 \\-0.3x_1 + x_2 - 7x_3 &= 9.5\end{aligned}$$

5.

$$\begin{aligned}-3x_2 + 7x_3 &= 2 \\x_1 + 2x_2 - x_3 &= 3 \\12x_1 + 2x_2 + 2x_3 &= 6\end{aligned}$$

6.

$$\begin{aligned}0.15x_1 + 2.11x_2 + 30.75x_3 &= -26.38 \\0.64x_1 + 1.21x_2 + 2.05x_3 &= 1.01 \\3.21x_1 + 1.53x_2 + 1.04x_3 &= 5.23\end{aligned}$$

7.

$$\begin{aligned}x_1 + x_2 - x_3 &= -3 \\6x_1 + 2x_2 + 2x_3 &= 2 \\-3x_1 + 4x_2 + x_3 &= 1\end{aligned}$$

8.

$$\begin{aligned}2x_1 + x_2 - x_3 &= 1 \\5x_1 + 2x_2 + 2x_3 &= -4 \\3x_1 + x_2 + x_3 &= 5\end{aligned}$$

9.

$$\begin{aligned}3x - 0.1y - 0.2z &= 7.85 \\0.1x + 7y - 0.3z &= -19.3 \\0.3x_1 - 0.2x_2 + 10x_3 &= 71.4\end{aligned}$$

10.

$$\begin{aligned}17x_1 - 2x_2 - 3x_3 &= 500 \\-5x_1 + 21x_2 - 2x_3 &= 200 \\-5x_1 - 5x_2 + 22x_3 &= 30\end{aligned}$$

5.40.5. Ejercicios

Ejercicio 7. *Aplicar el método de Jacobi para resolver los siguientes sistemas de ecuaciones lineales*

$$1. A = \left(\begin{array}{cccc|c} 10 & 2 & -1 & 0 & 26 \\ 1 & 20 & -2 & 3 & -15 \\ -2 & 1 & 30 & 0 & 53 \\ 1 & 2 & 3 & 20 & 47 \end{array} \right)$$

$$2. A = \left(\begin{array}{ccc|c} -1 & 2 & 10 & 11 \\ 11 & -1 & 2 & 12 \\ 1 & 5 & 2 & 8 \end{array} \right)$$

$$3. A = \left(\begin{array}{ccc|c} 8 & 2 & 3 & 51 \\ 2 & 5 & 1 & 23 \\ -3 & 1 & 6 & 20 \end{array} \right)$$

$$4. A = \left(\begin{array}{cccc|c} 2 & -1 & 1 & 3 & 10 \\ 2 & 2 & 2 & 2 & 1 \\ -1 & -1 & 2 & 2 & -5 \\ 3 & 1 & -1 & 4 & 6 \end{array} \right)$$

$$5. A = \left(\begin{array}{cccc|c} 3 & 1 & 1 & -1 & 5 \\ 0 & 2 & 1 & 4 & 0 \\ 1 & 1 & -1 & 9 & 1 \\ 2 & 4 & 6 & 3 & 0 \end{array} \right)$$

$$6. A = \left(\begin{array}{cccc|c} 10 & -1 & 2 & 0 & 6 \\ -1 & 11 & -1 & 3 & 25 \\ 2 & -1 & 10 & -1 & -11 \\ 0 & 2 & -1 & 8 & 15 \end{array} \right)$$

7.

$$x_1 + 2x_2 - 2x_3 = 7$$

$$x_1 + x_2 + x_3 = 2$$

$$2x_1 + 2x_2 + x_3 = 5$$

8.

$$-4x_1 + 14x_2 = 10$$

$$-5x_1 + 13x_2 = 8$$

$$-x_1 + 2x_3 = 1$$

9.

$$\begin{aligned}x + y + 2z &= 1 \\x + 2y + z &= 1 \\2x + y + z &= 1\end{aligned}$$

10.

$$\begin{aligned}6x_1 - 2x_2 + 2x_3 + 4x_4 &= 10 \\12x_1 - 8x_2 + 6x_3 + 10x_4 &= 20 \\3x_1 - 13x_2 + 9x_3 + 3x_4 &= 2 \\-6x_1 + 4x_2 + x_3 - 18x_4 &= -19\end{aligned}$$

6. Introducción al uso de R

6.1. Sesiones en RStudio

Al utilizar R, existen varios entornos que facilitan la gestión y ejecución de rutinas, *archivos con extensión .R*, el más popular es *RStudio* o bien directamente desde la terminal o ejecutando simplemente *R*, la ventaja de *RStudio* es que permite que en una pantalla podamos visualizar: Consola (lugar donde se ejecutan los comandos directamente), History (el histórico de las variables y funciones definidas mismo que puede guardarse para ser invocado posteriormente), Plots (ventana en la que se muestran los gráficos generados), Help (la ayuda sobre comandos, funciones, sintaxis en R), Files (lugar donde se manejan los archivos, es decir leer, guardar, mover o renombrar archivos), y Packages (espacio para instalar o cargar paquetes de manera gráfica), todo esto para facilitar el manejo y ejecución de rutinas compatibles con R. Por otra parte **Workspace** es un entorno en el que se incluyen todos los objetos definidos, al final de una sesión de R, este entorno puede guardarse una imagen del mismo para ser cargada posteriormente.

Durante el uso de R en ocasiones se requiere limpiar la consola, para esto al presionar **Ctrl+L**.

6.2. Uso de R

- **Constantes:** π , $\exp(1)$
- Las constantes pueden ser de tipo *integer*, *double* o *complex*, el tipo de constante se puede consultar con la función **typeof()**

```
> typeof('mi constante')  
[1] "character"
```

- Operadores: $<$, $>$, $>=$, $<=$, $!=$, (Not), $\parallel$ (OR), $\&$ (And), $==$ (comparar)
- Operadores aritméticos: $+$, $-$, $*$, $^$ potencia, $\%\%$ resto de la división entera, $\%/\%$ división entera.
- Logaritmos y exponenciales: **log** logaritmo natural, **log(x,b)** ($\log_b x$) y **exp(x)** (e^x).
- Funciones trigonométricas **cos(x)**, **sin(x)**, **tan(x)**, **acos(x)**, **asin(x)**, **atan(x)**.

- Funciones misceláneas `abs(x)`, `sqrt(x)`, `floor(x)`, `ceiling(x)`, `max(x)`, `sign(x)`.
- Comando `options(digits=k)`:

```
> 1/3.0
[1] 0.3333333
> options(digits=3)
> 1/3
[1] 0.333
> 1/17.0
[1] 0.0588
> options(digits=3)
> 1/17.0
[1] 0.0588
> options(digits=5)
> 1/17.0
[1] 0.058824
> options(digits=9)
> 1/17.0
[1] 0.0588235294
```

- Comando `round(x,n)` redondea x a n decimales, el valor por defecto es $n = 6$.
- Comando `cat('caracter1','caracter2')` concatena dos cadenas o valores y el resultado lo convierte a un objeto tipo *string*.

6.3. Funciones

```
nombrefun = function(a1,a2,...,an) {
# código ...
instruccion-1
instruccion-2
# ...
return( ... ) #valor que retorna (o también la última instrucción, si ésta retorna algo)
}
```

6.4. Clase en Laboratorio de Cómputo

Ejercicio 8. 1. Generar un archivo tipo *Rmd*, personalizar de manera básica

2. Realizar las siguientes operaciones

```
x = c(1.1, 1.2, 1.3, 1.4, 1.5)
x = 1:5
x = seq(1,3, by =0.5)
x = seq(5,1, by =-1)
x = rep(1, times = 5)
length(x)
rep(x, 2)
```

```
set.seed(123)
x = sample(1:10, 5)
```

```
3. x = 1:5
   y = rep(0.5, times = length(x))
   x+y
   x*y
   x^y
   1/(1:5)
```

```
4. x = 1:5
   2*x
   1/x^2
   x+3
```

```
5. A = matrix(rep(0,9), nrow = 3, ncol= 3);
   B = matrix(c(1,2,3,5,6,7), nrow = 2, byrow=T);
   x = 1:3; y = seq(1,2, by = 0.5); z = rep(8, 3) ; x; y; z
   C = matrix(c(x,y,z), nrow = length(x)); C # ncol no es necesario declararlo
   xi = seq(1,2, by 0.1); yi = seq(5,10, by = 0.5)
   rbind(xi,yi)
   cbind(xi,yi)
```

```
6. A = diag(c(3,1,3));
   diag(A)
   n = 3
   I = diag(1, n);
   D = diag(diag(A))
   J = diag(1, 3, 4);
```

```
7. B = matrix(c( 1, 1 ,8,
                 2, 0, 8,
                 3, 2, 8), nrow = 3, byrow=TRUE); B

   B[2, 3]
   B[3,]
   B[,2]
   B[1:2,c(2,3)]
```

```
8. A = matrix(c( 1, 1 ,8,
                 2, 0, 8,
                 3, 2, 8), nrow = 3, byrow=TRUE); A
   A[c(1,3), ] = A[c(3,1), ]
   A[2, ] = A[2, ] - A[2,1]/A[1,1]*A[1, ]
```

```

9. x = c(2, -6, 7, 8, 0.1, -8.5, 3, -7, 3)
   which.max(x)
   which.max(abs(x))

10.
   A = matrix(1:9, nrow=3); A # Por columnas
   B = matrix(rep(1,9), nrow=3); B

   A+B
   A*B
   A%%B
   A^2
   A-2
   3*A
   t(A)
   det(A)
   C <- A-diag(1,3); det(C)

11.

   notas = matrix(c(80, 40, 70, 30, 90, 67, 90,
                    40, 40, 30, 90, 100, 67, 90,
                    100,100,100, 100, 70, 76, 95), nrow=3, byrow=TRUE); notas
   # crear columna con la suma de los renglones
   #agregar la columna al final de la matriz

12.
   u=c(1,2)
   v=c(-2,3)
   w=c(3,-5)
   norma=function(u){sqrt(sum(u^2))}

13.
   t(u-2*v)%*%w
   norma(u+v+w)
   norma(u)+norma(v)+norma(w)
   t(u-v)%*%(v-w)

14.
   u=c(8,3);a=c(4,-5)
   ProyOrto=function(u,a){(t(u)%*%a)*a/norma(a)}
   ProyOrto(u,a)
   ProyOrto(c(2,1,-4),c(-5,3,11))

15.
   A=matrix(c(1,0,0,1/3,4,0,1/2,3,2),ncol=3,byrow=TRUE)
   B=matrix(c(9,0,0,0,1,8,0,0,0,-2,7,0,0,0,-3,6),ncol=4)

```

```

solve(A) # Inversa de A
det(A)
solve(B);det(B)

```

16.

```

A=matrix(c(4,4.001,4.001,4.002),ncol=2)
B=A;B[2,2]=4.002001
solve(A)
solve(B)

```

17.

```

X=matrix(runif(12),ncol=3)
u=runif(4)
A=t(X)%*%X
B=u%*%t(u)
DA=eigen(A)$values
TA=eigen(A)$vectors
t(TA)%*%TA
prod(DA);det(A)
qr(A)$rank # Rango de una matriz
sum(diag(DA)!=0)

```

18.

```

DB=eigen(B)$values
TB=eigen(B)$vectors
t(TB)%*%TB
prod(DB);det(B)
qr(B)$rank
sum(diag(DB)!=0)

```

19.

```

A=matrix(c(3,2,0,2,3,0,0,0,3),ncol=3)
eig_A=eigen(A)
eig_A$values
eigen(A%*%A)$values
eigen(solve(A))$values
eig_A$vectors%*%diag(eigen(A%*%A)$values)%*%t(eig_A$vectors);A%*%A

```

20.

```

A=matrix(c(6,10,1,10,6,5),ncol=2)
ginvMP=function(A){
  res=svd(A)
  res$v%*%diag(1/res$d)%*%t(res$u)}
B=ginvMP(A)
A%*%B%*%A

```

21.

```
es.positiva=function(A){
  if (ncol(A)!=nrow(A)) stop("Esto se hace para matrices cuadradas")
  v=eigen(A)$values
  tol=ncol(A)*max(abs(v))* .Machine$double.eps
  if (sum(v>tol)==length(v)) return(TRUE) else return(FALSE)}

A=matrix(c(2,-3/2,-3/2,3),ncol=2);es.positiva(A)
A=matrix(c(1,0.8,0.5,0.8,0.6,0.4,0.5,0.4,0.25),ncol=3);es.positiva(A)
```

22.

```
matrixA=function(m){
  A=matrix(c(1,-2,-2,m),ncol=2)
  return(A)}

dmatrixA=function(m){det(matrixA(m))}

m=seq(-4,10,len=101)
plot(m,mapapply(dmatrixA,m=m),type="l") # Dibuja el determinante en funci?n de m
abline(h=0)
A=matrixA(-2)
print(z<-eigen(A))
```

23.

```
A=cbind(c(3,1,0),c(1,3,0),c(0,0,3))
B=matrix(0,ncol=3,nrow=3);B[3,3]=2
eigen(A)
eigen(B)

eigen(A+B) # Trampilla
```

7. Apendice A: Breve historia de los Métodos Numéricos

Un *método numérico* es un proceso matemático *iterativo* cuyo objetivo es encontrar la aproximación a una solución específica con un cierto error previamente determinado. Los métodos numéricos requieren de una aproximación a la solución real al problema, misma que es corregida a través de la repetición de un cierto proceso que debe arrojar soluciones cada vez más cercanas al valor real. Cada corrección de un valor inicial se conoce como *iteración*. El proceso es controlado por medio de la medición de una cantidad de error predefinido entre dos aproximaciones sucesivas.

La historia de los métodos numéricos es la colección de acontecimientos matemáticos en los que se resuelven problemas sin el uso de la matemática analítica. Algunos de los métodos más utilizados en la actualidad fueron creados mucho antes de la invención de la computadora; su aplicación era extenuante y complicada porque cada iteración requería de una diversidad de operaciones aritméticas que se realizaban por grupos enteros de calculistas, evidentemente, de forma manual. La historia de los

métodos numéricos es paralela, al menos desde la mitad del siglo XIX, a la historia de la computación. Las contribuciones más actuales radican en la creación de software que minimiza los errores y mejora las aproximaciones de los resultados [1].

- 1650 a.C. Se crean los Papiros de Rhind en los que se escribe un método para resolver expresiones matemáticas sin álgebra.
- 250 a.C. Euclides crea el Método de Exhaustión, que consiste en aproximar figuras geométricas (triángulos, cuadrados, pentágonos, etc.) consecutivamente dentro de un círculo para obtener una aproximación a π .
- Siglo IX d.C. Al Juarismi crea los *algoritmos*.
- 1623. John Napier inventa los *huesos de Napier*, que son arreglos prácticos de logaritmos en tablas.
- Siglo XVII. Isaac Newton crea los procesos de interpolación polinomial.
- Siglo XVIII. Leibnitz crea el Cálculo diferencial.
- 1768. Euler crea soluciones aproximadas a ecuaciones diferenciales con el principio de la integración numérica. Jacob Stirling y Brook Taylor presentan el Cálculo de diferencias finitas.
- 1822. Charles Babbage inventa la *Máquina diferencial*.
- 1843. Ada, condesa de Lovelace, publica sus notas sobre la máquina analítica de Charles Babbage.
- 1890. (IBM) Tabula el censo estadounidense empleando las máquinas de tarjetas perforadas de Herman Hollerith.
- 1931. Vannevar Bush diseña el analizador diferencial, un computador analógico electromecánico. En 1945 publicará el artículo *Cómo podremos pensar* en el que describe la computación personal.
- 1937. Alan Turing publica *Sobre los números computables*, en el que describe un computador universal. En este mismo año, Howard Aiken propone la construcción de un gran computador y descubre partes de la máquina diferencial de Babbage en Harvard; también John Vincent Atanasoff conceptualiza el computador electrónico (la cual completará en 1939).
- 1938. William Hewlett y David Packard crean su empresa en Palo Alto, California, Estados Unidos.
- 1939. Turing comienza a descifrar los códigos secretos alemanes.
- 1944. John Von Neumann redacta el primer informe sobre EDVAC. En distintas universidades de Estados Unidos se desarrollan proyectos sobre computadoras cuya aplicación (secreta) será apoyar a la milicia en cálculos balísticos (ecuaciones diferenciales).
- 1950. Turing crea su famosa prueba sobre la inteligencia artificial; se suicidará en 1954. J.H. Wilkinson acudió al Laboratorio Nacional de Física de Reino Unido para construir una versión más simple de la máquina de Turing; construyó la *ACE (Automatic Computing Engine)* para resolver cálculos con matrices.
- 1953. John W. Backus, empleado de IBM, desarrolla *FORTRAN (Formulae Translating)*, como una alternativa al uso del lenguaje ensamblador; se usó por primera vez en una IBM 704.
- 1958. Se anuncia la creación de la Agencia de Proyectos de Investigación Avanzada (ARPA).

- 1962. Doug Engelbart publica *Aumentar el intelecto humano*; en 1963, junto con Bill English inventará el ratón.
- 1968. Noyce y Moore fundan *INTEL*.
- 1969. Misión Apolo 11. Katherine Johnson calcula la trayectoria del cohete Mercurio. Dorothy Vaughan se convierte en la supervisora de IBM dentro de la NASA. Mary Jackson es la primera ingeniera aeroespacial en Estados Unidos. Margaret Hamilton escribe el código del programa que controló la nave. Todas ellas tuvieron una participación fundamental para que la misión fuera un éxito.
- 1970. Investigadores visitantes en el *Argonne National Laboratory* de Estados Unidos traducen códigos de *ALGOL* para obtener eigenvalores planteados por Wilkinson para incluirlos en *FORTRAN*. De esta labor nace *EISPACK* en 1976 y posteriormente *LINPACK* en 1976.
- 1973. Vint Cerf y Bob Kahn completan los protocolos TCP/IP.
- 1975. Bill Gates y Paul Allen desarrollan el lenguaje de programación *BASIC*; fundan *Microsoft*. Steve Jobs y Steve Wozniak lanzan el *Apple I*.
- 1983. Richard Stallman empieza a desarrollar el proyecto *GNU*.
- 1986. Cleve Moler, a partir de *EISPACK* y *LINPACK*, crea *MATLAB*; funda la empresa *MathWorks*.
- 1991. Linus Torvalds lanza la primera versión de *Linux*. Tim Berbers-Lee anuncia la *World Wide Web*.

8. Programas y rutinas en R

Sesión 1

8.1. Operadores lógicos

```
17<5
17>5
17<=5
17>=5
17!=5
17==5
```

8.2. OPERADORES ARITMETICOS

8.2.1. SUMA, RESTA, MULTIPLICACION, DIVISION, POTENCIA, MODULO, DIVISION ENTERA

```
17+5
17*5
17*5
17^5
17%/5
17%%5
```


8.2.2. LOGARITMOS Y EXPONENCIALES

```
log(1)
log(12)
log(12,2)
exp(12)
exp(1)
```

8.2.3. FUNCIONES TRIGONOMETRICAS

```
sin(45)
cos(45)
tan(45)
asin(0.96)
acos(0.97)
atan(0.45)
```

8.2.4. FUNCIONES VARIAS

```
abs(-34)
sqrt(8)
floor(1.56)
ceiling(1.56)
max(4,7,2,12)
min(4,7,2,12)
sign(-45)
```

8.2.5. EJERCICIOS DE PRACTICA

```
# calcular la expresion cos(pi/6+pi/2)+e^2
# calcular la expresion cos(pi/6+pi/2)+e^2*log(5)+arc cos(1/raiz(2))
# introducir las siguientes expresiones:
# a) 1/7
# b) options(digits=3); 1/7
# c) options(digits=6); 1/7
# d) round(67.45)
# e) round(75.324568,2)
# f) options(digits=7);
# g) signif(56.345458234234,2)
# h) signif(56.345458234234)
# i) exp(-30)
# j) options(scipen= 999)
# k) exp(-30)
# l) options(scipen=0)
```

Sesión 2

8.3. EJERCICIOS DE PRACTICA

8.3.1. DEFINICION DE CONSTANTES

```
e = exp(1);  
x = 0.0034  
e <- exp(1)  
x <- 0.034;  
x0 = e^(2*x)
```

8.3.2. CONCATENAR Y PEGAR EXPRESIONES

```
txt = "El valor de x0 es _"  
cat(txt, x0)  
paste(txt,x0)  
paste0(txt,x0)
```

8.3.3. ASIGNACION E IMPRESION

```
x0 <- 1  
x1 <- x0 - pi*x0 + 1  
(x1 <- x0 - pi*x0 + 1 )  
print(x1)
```

8.3.4. LISTADO DE OBJETOS DEFINIDOS

```
ls()  
# Eliminar todos los objetos  
rm(list= ls())  
ls()
```

8.3.5. IMPRIMIR PEGAR AVANZADO

```
x0 <- 1  
x1 <- x0 - pi*x0 + 1  
cat("x0 =", x0, "\n", "x1 =", x1)
```

8.3.6. EJERCICIOS DE PRACTICA

Sesión 3

8.3.7. DEFINICION DE FUNCIONES

```
# nombre_funcion <- function(param1,param2,param3,...,paramn){  
# instruccion 1  
# instruccion 2  
# return(valor_de_retorno)  
#}
```

8.3.8. Ejemplo 1

```
fun1 <- function(x,a,b,h,k){  
  res <- a+b*cos(hx+k)  
  return(res)  
}
```

8.3.9. Ejemplo 2

```
Discriminante <- function(a,b,c){  
  res <- b^2-4*a*c  
  return(res)  
}
```

8.3.10. GRAFICAS

```
fun2 <- function(x,h,k){  
  res <- 1/h*sin(k*x)  
  return(res)  
}
```

```
f2 <- fun2(1:100,2,3)  
plot(f2,type="l", col= "red", lwd=2,  
     main= "Grafico de la funcion f2",  
     xlab= "x",  
     ylab="f(x)=1/h*sin(k*x)",  
     axes= TRUE)
```

8.3.11. EJEMPLOS DE PRACTICA

Graficar: rectas, parabolas, cubicas, polinomios, exponenciales, logaritmos

8.4. Introducción a Factorización LU

8.4.1. Inversa de una matriz

```
'''{r}  
A=matrix(c(1,3,-2,1,1,4,-1,2,0,1,-1,4,2,6,1,2),nrow = 4,byrow = TRUE)  
(A)  
Aorig<- A  
'''
```

Paso 1)

```
'''{r}  
I = diag(4);(I)  
'''
```

Paso 2)

```

'''{r}
AInv <- cbind(A,I); (AInv)
A <- AInv
'''

```

Paso 3)

```

'''{r}
121 <- A[2,1]/A[1,1];(121)
131 <- A[3,1]/A[1,1];(131)
141 <- A[4,1]/A[1,1];(141)
'''

```

Paso 4)

```

'''{r}
A[2,] <- A[2,]-121*A[1,];
A[3,] <- A[3,]-131*A[1,];
A[4,] <- A[4,]-141*A[1,];
(A)
'''

```

Paso 5)

```

'''{r}
Atmp <- A[2,]
A[2,] <- A[3,]
A[3,] <- Atmp
(A)
'''

```

Paso 6)

```

'''{r}
132 <- A[3,2]/A[2,2];(132)
142 <- A[4,2]/A[2,2];(142)
'''

```

Paso 7)

```

'''{r}
A[3,] <- A[3,]-132*A[2,];
A[4,] <- A[4,]-142*A[2,];
(A)
'''

```

Paso 8)

```

'''{r}
esc <- 1/A[3,3]
A[3,] <- esc*A[3,]
(A)
'''

```

Paso 9)

```

'''{r}
143 <- A[4,3]/A[3,3];(143)
'''

```

Paso 10)

```

'''{r}
A[4,] <- A[4,]-143*A[3,];
(A)
'''

```

Paso 11)

```

'''{r}
esc <- 1/A[4,4]
A[4,] <- esc*A[4,]
(A)
'''

```

Paso 12)

```

'''{r}
134 <- A[3,4]/A[4,4];(134); A[3,] <- A[3,]-134*A[4,]
124 <- A[2,4]/A[4,4];(124); A[2,] <- A[2,]-124*A[4,]
114 <- A[1,4]/A[4,4];(114); A[1,] <- A[1,]-114*A[4,]
(A)
'''

```

Paso 13)

```

'''{r}
123 <- A[2,3]/A[3,3];(123); A[2,] <- A[2,]-123*A[3,]
113 <- A[1,3]/A[3,3];(113); A[1,] <- A[1,]-113*A[3,]
(A)
'''

```

Paso 14)

```
'''{r}
112 <- A[1,2]/A[2,2];(112); A[1,] <- A[1,]-112*A[2,]
(A)
'''
```

Paso 15)

```
'''{r}
AInv <- A[,5:8]; (AInv)
'''
```

Paso 16)

```
'''{r}
IdCalc <- Aorig%*%AInv; (IdCalc)
'''
```

8.4.2. Factorización LU

Ejemplo 6. '''{r}

```
A=matrix(c(1,1,1,1,2,3,1,5,-1,1,-5,3,3,1,7,-2),byrow = TRUE,nrow = 4); (A)
'''
```

```
'''{r}
b=matrix(c(10,31,-2,18),nrow = 4);(b)
'''
```

```
'''{r}
Ab <- cbind(A,b); (Ab)
'''
```

Los pivotes se definen como $a_{kk}^{(k)}$, luego construimos los multiplicadores: $l_{i,k} = a_{ik}^{(k)}/a_{kk}^{(k)}$, $l_{21} = a_{21}/a_{11}$ y $l_{31} = a_{31}/a_{11}$ y $l_{41} = a_{41}/a_{11}$

```
'''{r}
A <- Ab;
121 <- A[2,1]/A[1,1];(121)
131 <- A[3,1]/A[1,1];(131)
141 <- A[4,1]/A[1,1];(141)
'''
```

Construimos la matriz U , donde las entradas $a_{ij}^{(k+1)} = a_{ik}^{(k)} - l_{ik} \times a_{kj}^{(k)}$

```
'''{r}
A[2,] <- A[2,]-121*A[1,]; (A[2,])
A[3,] <- A[3,]-131*A[1,]; (A[3,])
A[4,] <- A[4,]-141*A[1,]; (A[4,])
'''
```

Es decir la matriz resultante es:

```
'''{r}
(A)
'''
```

entonces el pivote es $A_{22} = 1$, calculemos los l_{32} y l_{42}

```
'''{r}
132 <- A[3,2]/A[2,2];(132)
142 <- A[4,2]/A[2,2];(142)
'''
```

Haciendo cero debajo del pivote

```
'''{r}
A[3,] <- A[3,]-132*A[2,];(A[3,])
A[4,] <- A[4,]-142*A[2,];(A[4,])
'''
```

La matriz A queda de la forma:

```
'''{r}
(A)
'''
```

por tanto el pivote es $A_{33} = -2$, y resta calcylar l_{43}

```
'''{r}
143 <- A[4,3]/A[3,3];(143)
'''
```

haciendo cero debajo del pivote

```
'''{r}
A[4,] <- A[4,]-143*A[3,];(A[4,])
'''
```

Por lo tanto la matriz A resultante es

```
'''{r}
(A)
'''
```

entonces podemos construir la matriz L con los valores l_{ij} calculados en los pasos anteriores

```
'''{r}
L=diag(4);(L)
'''
```

```
'''{r}
L[2,1] <- 121
L[3,1] <- 131
L[4,1] <- 141
L[3,2] <- 132
L[4,2] <- 142
L[4,3] <- 143
(L)
'''
```

```
'''{r}
U <- A[,1:4];(U)
'''
```

Verifiquemos que efectivamente $A = LU$

```
'''{r}
Acalculada <- L%*%U; (Acalculada)
'''
```

Ejemplo 7. *Apliquemos lo desarrollado para la siguiente matriz A*

```
'''{r}
A=matrix(c(4,0,1,1,3,1,3,1,0,1,2,0,3,2,4,1),nrow = 4,byrow = TRUE); (A)
'''
```

```
'''{r}
b=matrix(c(5,6,13,1),nrow = 4);(b)
'''
```

Paso 1)

```
'''{r}
Ab <- cbind(A,b)
A <- Ab;
'''
```

Paso 2)

```
'''{r}
121 <- A[2,1]/A[1,1];(121)
131 <- A[3,1]/A[1,1];(131)
141 <- A[4,1]/A[1,1];(141)
'''
```


Paso 3)

```
'''{r}
A[2,] <- A[2,]-121*A[1,]; (A[2,])
A[3,] <- A[3,]-131*A[1,]; (A[3,])
A[4,] <- A[4,]-141*A[1,]; (A[4,])
'''
```

Paso 4)

```
'''{r}
132 <- A[3,2]/A[2,2]; (132)
142 <- A[4,2]/A[2,2]; (142)
'''
```

Paso 5)

```
'''{r}
A[3,] <- A[3,]-132*A[2,]; (A[3,])
A[4,] <- A[4,]-142*A[2,]; (A[4,])
'''
```

Paso 6)

```
'''{r}
143 <- A[4,3]/A[3,3]; (143)
'''
```

Paso 7)

```
'''{r}
A[4,] <- A[4,]-143*A[3,]; (A[4,])
'''
```

Paso 8)

```
'''{r}
L=diag(4); (L)
'''
```

Paso 9)

```
'''{r}
L[2,1] <- 121
L[3,1] <- 131
L[4,1] <- 141
L[3,2] <- 132
```

```
L[4,2] <- 142
L[4,3] <- 143;
(L)
'''
```

Paso 10)

```
'''{r}
U <- A[,1:4];(U)
'''
```

Paso 11)

```
'''{r}
Acalculada <- L%*%U; (Acalculada)
'''
```

Ejercicio 9. *Aplicar la factorización LU a la matriz A definida por*

```
'''{r}
A=matrix(c(2,3,2,4,
           4,10,-4,0,
           -3,-2,-5,-2,
           -2,4,4,-7),nrow = 4,byrow = TRUE)
(A)
'''
```

8.4.3. Notas importantes

Nota 10. *Si se fija $l_{ii} = 1$ se le denomina factorización Doolittle.*

Nota 11. ***Nota 2** Una matriz A tiene factorización de Doolittle si y sólo si se le puede aplicar el método de eliminación de Gauss sin pivoteo.*

Nota 12. *Si $U = L^T$ entonces la factorización se le denomina factorización de Cholesky.*

Nota 13. *Si la matriz es simétrica y definida positiva, entonces $A = LL^T$.*

8.5. Métodos de Eliminación directa

8.5.1. Gaussiana Simple

```
'''{r,echo=FALSE}
gauss_simple <- function(A, b, tolerancia, verbose = FALSE) {
  if (!is.matrix(A)) stop("A debe ser una matriz.")
  if (!is.numeric(b)) stop("b debe ser numérico.")
  n <- nrow(A)
  if (ncol(A) != n) stop("A debe ser cuadrada.")
  if (length(b) != n) stop("Longitud de b debe ser igual al número de filas de A.")
}
```

```

Ab <- cbind(A, b);
numcols = ncol(Ab)
for (k in 1:(n - 1)) {
  pivote <- Ab[k, k]
  if (abs(pivote) < tolerancia) {
    stop(sprintf("Pivote casi cero en fila %d (%.3e).
    El método sin pivoteo falla.", k, pivote))
  }
  if (k + 1 <= n) {
    for (i in (k + 1):n) {
      m <- Ab[i, k] / pivote;
      Ab[i, k:numcols] <- Ab[i, k:numcols] - m * Ab[k, k:numcols]
      if (abs(Ab[i, k]) < tolerancia) Ab[i, k] <- 0
      if (verbose) {
        cat(sprintf("k=%d, i=%d, m=%.6g\n", k, i, m));
        print(Ab)}
    }
  }
}
x <- numeric(n)
for (i in n:1) {
  if (i == n) {
    suma <- 0
  } else {suma <- sum(Ab[i, (i + 1):n] * x[(i + 1):n])}
  x[i] <- (Ab[i, n + 1] - suma) / Ab[i, i]
}
list(x=x,U=Ab[, 1:n],Ab=Ab)
}
'''

```

Ejemplo 8.

```

'''{r,echo=FALSE}
A <- matrix(c(
  2, 1, -1,
  -3, -1, 2,
  -2, 1, 2
), nrow = 3, byrow = TRUE)
b <- c(8, -11, -3);
tolerancia <- 1e-12;
res <- gauss_simple(A, b,tolerancia);
res <- gauss_simple(A, b,tolerancia, verbose = TRUE)
res$x;res$U;res$Ab;A %% res$x
'''

```

8.5.2. Gaussiana con pivoteo parcial

```

'''{r}
gauss_piv_parcial <- function(A, b, tolerancia, verbose = FALSE) {

```

```

if (!is.matrix(A)) stop("A debe ser una matriz.")
n <- nrow(A);
if (ncol(A) != n) stop("A debe ser cuadrada.")
if (length(b) != n) stop("Dimensiones de b incorrectas.")
Ab <- cbind(A, b)
for (k in 1:(n-1)) {
  # Selección del pivote (máximo en valor absoluto en la
  #columna k desde la fila k)
  max_row <- which.max(abs(Ab[k:n, k])) + (k - 1)
  if (abs(Ab[max_row, k]) < tolerancia){
    stop(sprintf("Pivote casi nulo en columna %d", k))}
  # Intercambio de filas si es necesario
  if (max_row != k) {
    Ab[c(k, max_row), ] <- Ab[c(max_row, k), ]
    if (verbose) {
      cat(sprintf("Intercambio de fila %d con fila %d\n", k, max_row));
      print(Ab)}
    }
  for (i in (k + 1):n) {
    m <- Ab[i, k] / Ab[k, k];
    Ab[i, k:ncol(Ab)] <- Ab[i, k:ncol(Ab)] - m * Ab[k, k:ncol(Ab)]
    if (abs(Ab[i, k]) < tolerancia) Ab[i, k] <- 0
    if (verbose) {
      cat(sprintf("k=%d, i=%d, m=%.6g\n", k, i, m));print(Ab)}
    }
  }
}
x <- numeric(n)
for (i in n:1) {
  if (i == n) {
    suma <- 0
  } else {
    suma <- sum(Ab[i, (i + 1):n] * x[(i + 1):n])
  }
  x[i] <- (Ab[i, n + 1] - suma) / Ab[i, i]
}
list(x = x,U = Ab[, 1:n],Ab = Ab)
}
'''

```

Ejemplo 9.

```

'''{r,echo=FALSE}
A <- matrix(c(
  0, 2, 1,
  1, -2, -3,
  -1, 1, 2), nrow = 3, byrow = TRUE)
b <- c(3, -3, -1);
tolerancia <- 1e-12
res <- gauss_piv_parcial(A, b, tolerancia, verbose = TRUE);
res$x

```

'''

8.5.3. Gauss Jordan

```
'''{r}
gauss_jordan <- function(A, b, tolerancia, verbose = FALSE) {
  if (!is.matrix(A)) stop("A debe ser una matriz.")
  n <- nrow(A);
  m <- ncol(A)
  if (!is.null(b) && length(b) != n)
    stop("Dimensiones incompatibles entre A y b.")
  Ab <- cbind(A, as.matrix(b));
  ncols <- ncol(Ab); row <- 1
  for (col in 1:m) {
    if (row > n) break
    pivot_row_rel <- which.max(abs(Ab[row:n, col]))
    pivot_row <- row + pivot_row_rel - 1
    if (abs(Ab[pivot_row, col]) < tolerancia) {
      if (verbose) cat(sprintf("Sin pivote usable en col=%d (|piv < tol). Se omite columna.\n",
        next
    }
    if (pivot_row != row) {
      Ab[c(row, pivot_row), ] <- Ab[c(pivot_row, row), ]
      if (verbose) {
        cat(sprintf("Swap filas %d <-> %d (col=%d)\n", row,
          pivot_row, col));
        print(Ab)}
    }
    pivote <- Ab[row, col];
    Ab[row, ] <- Ab[row, ] / pivote
    if (verbose) {
      cat(sprintf("Normaliza fila %d por pivote %.6g (col=%d)\n",
        row, pivote, col));
      print(Ab)}
    for (r in 1:n) {
      if (r == row) next
      factor <- Ab[r, col]
      if (abs(factor) > tolerancia) {
        Ab[r, ] <- Ab[r, ] - factor * Ab[row, ]
        if (verbose) {
          cat(sprintf("R%d := R%d - (%.6g)*R%d (col=%d)\n",
            r, r, factor, row, col));print(Ab)}
      }
    }
    row <- row + 1
  }
  Ab[abs(Ab) < tolerancia] <- 0;
  out <- list(RREF = Ab);
```

```

X <- Ab[, (m + 1):ncols, drop = FALSE]
out$x <- if (ncol(X) == 1) as.vector(X) else X
return(out)
}
'''

```

Ejemplo 10. `'''{r,echo=FALSE}`

```

A <- matrix(c(
  2, 1, -1,
  -3, -1, 2,
  -2, 1, 2), nrow = 3, byrow = TRUE)
b <- c(8, -11, -3);
tolerancia <- 1e-12;
res <- gauss_jordan(A, b, tolerancia, verbose = TRUE);
res$x
'''

```

8.5.4. Cálculo de Inversa

```

'''{r}
gauss_jordan_inversa <- function(A,tolerancia, verbose = FALSE) {
  if (!is.matrix(A)) stop("A debe ser una matriz.")
  n <- nrow(A); m <- ncol(A)
  if (n != m) stop("Para invertir, A debe ser cuadrada.")
  Ab <- cbind(A, diag(n));
  ncols <- ncol(Ab);
  row <- 1
  for (col in 1:m) {
    if (row > n) break
    pivot_row_rel <- which.max(abs(Ab[row:n, col]))
    pivot_row <- row + pivot_row_rel - 1
    if (abs(Ab[pivot_row, col]) < tolerancia) {
      stop(sprintf("No hay pivote en la columna %d (|piv|<tol).",
        col))
    }
    if (pivot_row != row) {
      Ab[c(row, pivot_row), ] <- Ab[c(pivot_row, row), ]
      if (verbose) {
        cat(sprintf("Intercambia filas %d <-> %d (col=%d)\n",
          row, pivot_row, col));
        print(Ab)
      }
    }
    pivote <- Ab[row, col];
    Ab[row, ] <- Ab[row, ] / pivote
    if (verbose) {
      cat(sprintf("Normaliza fila %d por pivote %.6g (col=%d)\n",
        row, pivote, col));
      print(Ab)}
  }
}
'''

```

```

for (r in 1:n) {
  if (r == row) next
  factor <- Ab[r, col]
  if (abs(factor) > tolerancia) {
    Ab[r, ] <- Ab[r, ] - factor * Ab[row, ]
    if (verbose){
      cat(sprintf("R%d := R%d - (%.6g)*R%d (col=%d)\n",
        r, r, factor, row, col));
      print(Ab)
    }
  }
}
row <- row + 1
}
Ab[abs(Ab) < tolerancia] <- 0;
LHS <- Ab[, 1:m, drop = FALSE]
if (!all(LHS == diag(n))) {
  stop("La matriz es singular o la tolerancia es muy estricta.")
}
Ainv <- Ab[, (m + 1):ncols, drop = FALSE]
list(Ainv = Ainv, RREF = Ab)
}
'''

```

Ejemplo 11. `'''{r,echo=FALSE}`

```

tolerancia <- 1e-12
A <- matrix(c(
  1, 2, 3,
  0, 1, 4,
  5, 6, 0), nrow = 3, byrow = TRUE)
res <- gauss_jordan_inversa(A, tolerancia,
  verbose = TRUE);
res$Ainv;
round(A %*% res$Ainv, 6)
'''

```

8.5.5. Factorización LU

Sin Pivoteo

```

'''{r}
lu_simple <- function(A, tol = 1e-12, verbose = FALSE) {
  if (!is.matrix(A)) stop("A debe ser una matriz.")
  n <- nrow(A); m <- ncol(A)
  if (n != m) stop("A debe ser cuadrada.")
  U <- A;
  L <- diag(n)
  for (k in 1:(n - 1)) {
    piv <- U[k, k]

```

```

    if (abs(piv) < tol) {
      stop(sprintf("Pivote casi cero en k=%d. No se puede continuar.",
k))
    }
    for (i in (k + 1):n) {
      m <- U[i, k] / piv;
      L[i, k] <- m;
      U[i, k:n] <- U[i, k:n] - m * U[k, k:n]
      if (verbose) {
        cat(sprintf("k=%d, i=%d, m=%.6g\n", k, i, m));
        print(U)
      }
    }
  }
  list(L = L, U = U)
}
'''

```

Ejemplo 12. `''{r,echo=FALSE}`

```

A <- matrix(c(
  2,  1, -1,
 -3, -1,  2,
 -2,  1,  2
), 3, 3, byrow = TRUE)
lu <- lu_simple(A);lu$L; lu$U
'''

```

Con Pivoteo

```

''{r}
lu_piv_parcial <- function(A, tol = 1e-12, verbose = FALSE) {
  if (!is.matrix(A)) stop("A debe ser una matriz.")
  n <- nrow(A); m <- ncol(A);
  if (n != m) stop("A debe ser cuadrada.")
  U <- A;
  L <- diag(n);
  P <- diag(n)
  for (k in 1:(n - 1)) {
    max_row <- which.max(abs(U[k:n, k])) + (k - 1)
    if (abs(U[max_row, k]) < tol) {
      stop(sprintf("Matriz singular (pivote ~ 0).", k))
    }
    if (max_row != k) {
      U[c(k, max_row), ] <- U[c(max_row, k), ];
      P[c(k, max_row), ] <- P[c(max_row, k), ]
      if (k > 1) {
        L[c(k, max_row), 1:(k - 1)] <- L[c(max_row, k), 1:(k - 1)]
      }
    }
  }
}

```



```

    if (verbose) {
      cat(sprintf("Intercambia filas %d <-> %d\n",
        k, max_row));
      print(U)
    }
  }
  for (i in (k + 1):n) {
    m <- U[i, k] / U[k, k];
    L[i, k] <- m;
    U[i, k:n] <- U[i, k:n] - m * U[k, k:n];
    if (verbose) {
      cat(sprintf("k=%d, i=%d, m=%.6g\n", k, i, m));
      print(U)
    }
  }
}
list(P = P, L = L, U = U)
}
'''

```

Ejemplo 13. `'''{r,echo=FALSE}`

```

A <- matrix(c(
  0, 2, 1,
  1, -2, -3,
  -1, 1, 2), 3, 3, byrow = TRUE)
lu <- lu_piv_parcial(A, verbose = FALSE);
lu$P;
lu$L;
lu$U
'''

```

Solucion vía LU

```

'''{r}
forward_sub <- function(L, b) {
  n <- nrow(L);
  y <- numeric(n)
  for (i in 1:n) {
    y[i] <- (b[i] - sum(L[i, 1:(i - 1)] * y[1:(i - 1)]))
  }
  y
}
'''

```

```

'''{r}
# x en Ux = y
back_sub <- function(U, y, tol = 1e-12) {
  n <- nrow(U);

```

```

x <- numeric(n)
for (i in n:1) {
  s <- if (i == n) 0 else sum(U[i, (i + 1):n] * x[(i + 1):n])
  if (abs(U[i, i]) < tol) stop(sprintf("Pivote ~0 en U[%d,%d].",
    i, i))
  x[i] <- (y - s) / U[i, i]
}
x
}
'''

```

Resolucion sin pivoteo

```

'''{r}
solve_lu_simple <- function(A, b, tol = 1e-12) {
  lu <- lu_simple(A, tol = tol);
  y <- forward_sub(lu$L, b)
  x <- back_sub(lu$U, y, tol = tol)
  x
}
'''

```

8.5.6. Resolucion con pivoteo

```

'''{r}
solve_lu_piv_parcial <- function(A, b, tol = 1e-12) {
  lu <- lu_piv_parcial(A, tol = tol);
  Pb <- lu$P %*% b
  y <- forward_sub(lu$L, as.vector(Pb))
  x <- back_sub(lu$U, y, tol = tol)
  x
}
'''

```

Ejemplo 14. '''{r,echo=FALSE}

```

A <- matrix(c(
  0, 2, 1,
  1, -2, -3,
  -1, 1, 2), 3, 3, byrow = TRUE)
b <- c(3, -3, -1);
x <- solve_lu_piv_parcial(A, b);
x
'''

```

8.5.7. Factorización de Cholesky

```

'''{r}
tolerancia <- 1e-12
cholesky_fact <- function(A,tolerancia) {

```

```

if (!is.matrix(A)) stop("A debe ser una matriz.")
n <- nrow(A)
if (ncol(A) != n) stop("A debe ser cuadrada.")
if (!all(abs(A - t(A)) < tolerancia)) stop("A debe ser simétrica.")
L <- matrix(0, n, n)
for (j in 1:n) {
  suma <- sum(L[j, 1:(j-1)]^2)
  val <- A[j, j] - suma
  if (val <= 0) stop("A no es definida positiva (falló Cholesky).")
  L[j, j] <- sqrt(val)
  if (j < n) {
    for (i in (j+1):n) {
      suma <- sum(L[i, 1:(j-1)] * L[j, 1:(j-1)])
      L[i, j] <- (A[i, j] - suma) / L[j, j]
    }
  }
  cat(sprintf("Paso j=%d\n", j))
  print(L)
}
return(L)
}
'''

```

Ejemplo 15. `'''{r,echo=FALSE}`

```

# Matriz simétrica definida positiva
A <- matrix(c(
  4, 12, -16,
  12, 37, -43,
  -16, -43, 98), nrow = 3, byrow = TRUE)

L <- cholesky_fact(A,tolerancia)
L
'''

```

8.5.8. Solución vía Cholesky

```

'''{r}
tolerancia <- 1e-12
solve_cholesky <- function(A, b, tolerancia) {
  L <- cholesky_fact(A, tolerancia)
  y <- forward_sub(L, b)
  x <- back_sub(t(L), y, tolerancia)
  x
}
'''

```

Ejemplo 16.

```

'''{r,echo=FALSE}
b <- c(1, 2, 3)

```

```
x <- solve_cholesky(A, b,tolerancia)
x
A %*% x
'''
```

8.6. Métodos Iterativos

8.6.1. Gauss Jacobi

```
'''{r}
jacobi <- function(A, b, x0 = NULL, tol = 1e-8, maxiter = 1000) {
  if (!is.matrix(A)) stop("A debe ser una matriz.")
  n <- nrow(A)
  if (ncol(A) != n) stop("A debe ser cuadrada.")
  if (length(b) != n) stop("Dimensiones de b incompatibles.")
  if (is.null(x0)) x0 <- rep(0, n)
  x <- x0;
  D <- diag(diag(A));
  R <- A - D;
  for (k in 1:maxiter) {
    x_new <- (b - R %*% x) / diag(D)
    cat(sprintf("Iter %d: %s\n", k,
      paste(round(x_new, 6), collapse = " ")))
    if (sqrt(sum((x_new - x)^2)) < tol) {
      return(list(x = as.vector(x_new),
        iter = k, convergencia = TRUE))}
    x <- x_new
  }
  list(x = as.vector(x),
    iter = maxiter,
    convergencia = FALSE)
}
'''
```

Ejemplo 17.

```
'''{r,echo=FALSE}
A <- matrix(c(
  4, -1, 0,
  -1, 4, -1,
  0, -1, 3), 3, 3, byrow = TRUE)

b <- c(15, 10, 10)
res_jacobi <- jacobi(A, b, x0 = c(0, 0, 0), tol = 1e-8);
res_jacobi$x
'''
```

8.6.2. Gauss Seidel

```
'''{r}
gauss_seidel <- function(A, b, x0 = NULL,
tol = 1e-8, maxiter = 1000, verbose = FALSE) {
  if (!is.matrix(A)) stop("A debe ser una matriz.")
  n <- nrow(A)
  if (ncol(A) != n) stop("A debe ser cuadrada.")
  if (length(b) != n) stop("Dimensiones de b incompatibles.")
  x <- if (is.null(x0)) rep(0, n) else as.numeric(x0)
  if (length(x) != n) stop("Dimensión de x0 incompatible con A.")
  for (k in 1:maxiter) {
    x_old <- x
    for (i in 1:n) {
      aii <- A[i, i]
      if (abs(aii) < .Machine$double.eps) {
        stop(sprintf("A[%d,%d] = 0: Gauss{Seidel No se puede aplicar",
          i, i)))}
      s1 <- if (i > 1) sum(A[i, 1:(i - 1)] * x[1:(i - 1)]) else 0
      s2 <- if (i < n) sum(A[i, (i + 1):n] * x_old[(i + 1):n]) else 0
      x[i] <- (b[i] - s1 - s2) / aii
    }
    dx <- sqrt(sum((x - x_old)^2));
    res <- sqrt(sum((b - A %*% x)^2))
    cat(sprintf("Iter %4d |dx|=%.3e |res|=%.3e x=%s\n",
      k, dx, res, paste(round(x, 6), collapse=" ")))
    if (dx < tol && res < tol) {
      return(list(x = as.vector(x), iter = k,
        convergencia = TRUE,delta = dx, residuo = res))
    }
  }
  list(x = as.vector(x), iter = maxiter, convergencia = FALSE,
    delta = sqrt(sum((x - x)^2)),
    residuo = sqrt(sum((b - A %*% x)^2)))
}
'''
```

Ejemplo 18.

```
'''{r,echo=FALSE}
A <- matrix(c(
  4, -1, 0,
  -1, 4, -1,
  0, -1, 3
), 3, 3, byrow = TRUE)
b <- c(15, 10, 10)
res_gs <- gauss_seidel(A, b, x0 = c(0,0,0), tol = 1e-8);
res_gs$x
'''
```

Referencias

- [1] Isaacson, W. (2014). *Los innovadores: Los genios que inventaron el futuro*. Debate.